1 Algorithm

1.1 LIS

```cpp
// longest increasing subsequence
int LIS(const vector<int>& s) {
// use binary search to update the smallest element
// that ends in a subsequence of length d
    vector<int> v{};
    v.push_back(s[0]);
    for (int i{1}; i < s.size(); ++i)
        if (s[i] > v.back()) v.push_back(s[i]);
        else *lower_bound(v.begin(), v.end(), s[i]) = s[i];
    return v.size();
}
```

1.2 LCS

```cpp
// longest common subsequence
int LCS(const vector<int>& a, const vector<int>& b) {
    vector<vector<int>> dp(a.size() + 1, vector<int>(b.size() + 1));
    for (int i{1}; i <= int(a.size()); ++i)
        for (int j{1}; j <= int(b.size()); ++j) {
            dp[i][j] = max(dp[i][j], max(dp[i - 1][j], dp[i][j - 1]));
            if (a[i - 1] == b[j - 1]) dp[i][j] = max(dp[i][j], dp[i - 1][j - 1] + 1);
        }
    return dp[a.size()][b.size()];
}
```

2 Basic

2.1 mod helper function

```cpp
int add(int i, int j) {
    if ((i += j) >= MOD)
        i -= MOD;
    return i;
}

int sub(int i, int j) {
    if ((i -= j) < 0)
        i += MOD;
    return i;
}
```

2.2 self-defined-pq-operator

```cpp
auto cmp = [](int a, int b) {
    return a > b;
};
priority_queue<int, vector<int>, decltype(cmp)> pq(cmp);
```

2.3 generating all subsets

```cpp
for (int b = 0; b < (1<<n); b++) {
   vector<int> subset;
   for (int i = 0; i < n; i++) {
       if (b&(1<<i)) subset.push_back(vc[i]);
   }
}
```

2.4 tuple

```cpp
int main() {
    tuple<int, double, string> myTuple(100, 3.14, "hello world");
    tuple<int, double, string> anotherTuple = make_tuple(100, 18, "Tom");
    cout << get<0>(myTuple) << "\n"; // 取得第一個元素
    auto [a, b, c] = myTuple;
    get<1>(myTuple) = 5.43; // 修改第二個元素的值

    int a;
    double b;
    string c;
    tie(a, b, c) = myTuple;
}
```

2.5 memset

```cpp
memset(a, 0, sizeof(a)); // 0
memset(a, 0x3f3f3f3f , sizeof(a)); // INF
```

2.6 submask enumeration

```cpp
// O(3^n)

int m = 0b10110; // binary: 10110, decimal: 22

cout << "Mask: " << bitset<5>(m) << " (" << m << ")\\n";

// Enumerate all submasks
for (int s = m; s; s = (s - 1) & m) {
    cout << bitset<5>(s) << " (" << s << ")\\n";
}

// Optionally include the empty submask
cout << bitset<5>(0) << " (0)\\n";
```

2.7 stringstream split by comma

```cpp
while (std::getline(ss, segment, ',')) {
    segments.push_back(segment);
}
```

3 DP

3.1 walk

```cpp
/*
DP on graphs - O(N^3 log K)
給定一個簡單的有向圖 G，具有 N 個頂點，編號為 1, 2, …, N。

對於任意 i, j (1 ≤ i, j ≤ N)，給定整數 a_{i, j}，表示是否存在從頂點 i 指向頂點 j 的有向邊。若 a_{i,j} = 1，則存在邊；若 a_{i, j} = 0，則不存在。

求圖中長度為 K 的不同有向路徑數目，對 10^9+7 取模。路徑可重複通過相同邊（即允許重複邊）。

注意：當我們將鄰接矩陣 m 與 m 相乘時，得到的是長度為 2 的路徑數；若取 m 的 p 次方 m^p，則其 (i, j) 元素表示從 i 到 j 的長度為 p 的路徑數。
*/

void solve() {
    int n, k;
    cin >> n >> k;
    vector<vector<int>> m(n, vector<int>(n));

    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            cin >> m[i][j];
        }
    }

    Matrix<int> mat(m);
    mat = power(mat, k);
    int ans = 0;
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            ans += mat.mat[i][j];
            ans %= MOD;
        }
    }
    cout << ans << "\n";
}
```

3.2 grouping

```cpp
/*
狀壓 DP - O(3^N * + 2^N * N^2)
有 N 隻兔子，編號為 1,2,…,N。

對於每一對 i,j (1≤i,j≤N)，兔子 i 與 j 的相容度由整數 a_i,j 描述。這裡 a_i,i = 0 對於每個 i (1≤i≤N)，且 a_i,j = a_j,i 對於任意 i 與 j (1≤i,j≤N)。

A 將 N 隻兔子分成若干個群組。每隻兔子必須且僅屬於一個群組。分群後，對於每一對 i 與 j (1≤i<j≤N)，若兔子 i 與 j 屬於同一群組，A 即可獲得 a_i,j 分。

求 A 能獲得的最大總分。

令 cost[S] 表示將集合 S 中的所有兔子放在同一群組時所得到的分數。此值可在 O(2^N * N^2) 時間內計算。

接著我們計算 dp[S]，表示對集合 S 中的兔子進行分群時所能得到的最大分數。
*/

void solve() {
    int n;
    cin >> n;
    vector<vector<int>> a(n, vector<int>(n));
    vector<int> cost(1<<n, 0);
    vector<int> dp(1<<n, 0);

    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            cin >> a[i][j];
        }
    }

    // backtrack all subset
    for (int b = 0; b < (1<<n); ++b) {
        vector<int> subset;
        for (int i = 0; i < n; ++i) {
            if (b & (1<<i)) {
                for (const int& j : subset) {
                    cost[b] += a[i][j];
                }
                subset.pb(i);
            }
        }
    }
```

```
    // dp
    for (int i = 0; i < (1<<n); ++i) {
        int j = ((1<<n) - 1) ^ i;
        for (int s = j; s != 0; s = (s - 1)
             & j) {
            dp[i ^ s] = max(dp[i ^ s], dp[i]
                 + cost[s]);
        }
    }
    cout << dp[(1<<n) - 1] << "\n";
}
```

3.3 matching

```
/*
Bitmask DP - O(N * 2^N)
有 N 個男人和 N 個女人，分別編號為 1, 2, …, N。

對於每個 i, j (1 ≤ i, j ≤ N)，男人 i 和女人 j 的相容性由整數 a[i][j] 給出。
如果 a[i][j] = 1，則男人 i 和女人 j 是相容的；
如果 a[i][j] = 0，則不是。

A 正在嘗試組成 N 對，每對由一個相容的男人和女人組成。在這裡，每個男人和每個女人必須恰好屬於一對。

求 A 可以組成 N 對的方法數，結果對 10^9 + 7 取模。

定義 dp[S] 為將集合 S 中的女性與前 |S| 個男性配對的方法數。
*/

const int maxn = 21;
const int MOD = 1e9 + 7;
int n;
int grid[maxn][maxn];
int dp[1 << maxn];

void solve() {
    cin >> n;
    memset(dp, 0, sizeof(dp));

    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            cin >> grid[i][j];
        }
    }

    dp[0] = 1;
    for (int s = 0; s < (1 << n); ++s) {
        int ps = __builtin_popcount(s);
        for (int w = 0; w < n; ++w) {
            if ((s & (1 << w)) || !grid[ps][
                 w]) {
                continue;
            }
            dp[s | (1 << w)] += dp[s];
            dp[s | (1 << w)] %= MOD;
        }
    }
    cout << dp[(1 << n) - 1] << "\n";
}
```

3.4 projects

```
/*
LIS DP - O(N log N)
有 n 個你可以參加的專案。對於每個專案，你知道其開始與結束天數以及可獲得的報酬金額。
在同一天你最多只能參加一個專案。
問：你最多可以賺到多少金額？

dp[i] = 在第 i 天之前我們可以賺到的最大金額。
*/

void solve() {
    int n;
    cin >> n;
    vector<array<int, 3>> vc(n);
    map<int, int> days;

    for (int i = 0; i < n; ++i) {
        int a, b, p;
        cin >> a >> b >> p;
        days[a] = days[b] = 1;
        vc[i] = {a, b, p};
    }
    int idx = 1;
    for (auto& x : days) {
        x.second = idx++;
    }
    vector<int> dp(idx, 0);

    sort(vc.begin(), vc.end(), [](const
         array<int, 3>& va, const array<int,
         3>& vb) {
        if (va[1] != vb[1]) return va[1] <
             vb[1];
        if (va[0] != vb[0]) return va[0] <
             vb[0];
        return va[2] > vb[2];
    });

    int i = 0;
    for (int d = 1; d < idx; ++d) {
        dp[d] = dp[d - 1];
        while (i < n && days[vc[i][1]] == d)
             {
            dp[d] = max(dp[d], dp[days[vc[i
                 ][0]] - 1] + vc[i][2]);
            i++;
        }
    }
    cout << dp[idx - 1] << "\n";
}
```

3.5 elevator rides

```
/*
狀壓 DP - O(2^N)
有 n 個人想要搭電梯到樓頂，建築物只有一部電梯。你知道每個人的體重以及電梯的最大允許載重。最少需要搭乘多少次電梯？

定義 dp[S] = {r, w}，其中 r 是將集合 S 中的所有人送到樓頂所需的最少電梯次數，w 是最後一次電梯所載人的總重量。
*/

void solve() {
    int n, x;
    cin >> n >> x;
    vector<int> w(n);
    vector<pii> dp(1<<n, {INF, INF});

    for (int i = 0; i < n; ++i) {
        cin >> w[i];
    }

    dp[0] = {1, 0};
    for (int b = 1; b < (1<<n); ++b) {
        for (int i = 0; i < n; ++i) {
            if (b & (1<<i)) {
                auto [r_prev, w_prev] = dp[b
                     ^ (1<<i)];
                pii can;
                if (w_prev + w[i] <= x) {
                    can = {r_prev, w_prev +
                         w[i]};
                }
                else {
                    can = {r_prev + 1, w[i
                         ]};
                }
                dp[b] = min(dp[b], can);
            }
        }
    }
    cout << dp[(1<<n) - 1].first << "\n";
}
```

3.6 digit sum

```
/*
Digit DP - O(|K| * D)
計算在 1 到 K（含）之間，滿足其十進位數字和為 D 的倍數的整數數量，答案對 10^9+7 取模。

令 dp[i][j] 表示在已確定前 i 位數字的情況下，構成長度為 |K| 的數字且目前數字和 mod D 等於 j 的方法數。
*/

const int MOD = 1e9 + 7;
int dp[10001][101][2];

void solve() {
    string K;
    int D;
    cin >> K >> D;
    int len = K.size();
    memset(dp, 0, sizeof(dp));
    dp[0][0][1] = 1;

    for (int i = 1; i <= len; ++i) {
        int limit = K[i - 1] - '0';
        for (int s = 0; s < D; ++s) {
            for (int flag = 0; flag <= 1; ++
                 flag) {
                int ways = dp[i - 1][s][flag
                     ];
                if (ways == 0) continue;
                int max_d = (flag ? limit :
                     9);
                for (int d = 0; d <= max_d;
                     ++d) {
                    int rs = (s + d) % D;
                    int rflag = (flag && d
                         == max_d ? 1 : 0);
                    dp[i][rs][rflag] += ways
                         ;
                    dp[i][rs][rflag] %= MOD;
                }
            }
        }
    }

    int ans = (dp[len][0][0] + dp[len
         ][0][1]) % MOD;
    ans = (ans - 1 + MOD) % MOD;
    cout << ans << "\n";
}
```

3.7 sushi

```
/*
期望值 DP - O(N^3)
有 N 盤壽司，編號從 1 到 N。第 i 盤最初有 $a_i$ ($1 \le a_i \le 3$) 塊壽司。

太郎不斷擲一顆編號 1 到 N 的骰子。如果結果是第 i 盤且該盤還有壽司，他就吃掉一塊；否則什麼也不做。

請求出吃完所有壽司所需擲骰子的期望次數。

dp[x][y][z] 代表還有
x 盤剩 1 塊壽司、
y 盤剩 2 塊壽司、
z 盤剩 3 塊壽司時的期望擲骰次數。
*/

const int maxn = 301;
double dp[maxn][maxn][maxn];
int n;
```

```
double dfs(int x, int y, int z) {
    if (x < 0 || y < 0 || z < 0) return 0;
    if (x == 0 && y == 0 && z == 0) return
         0;
    if (dp[x][y][z] > 0) return dp[x][y][z];
    double ans = n + x * dfs(x - 1, y, z)
                   + y * dfs(x + 1, y - 1, z
                        )
                   + z * dfs(x, y + 1, z -
                        1);
    return dp[x][y][z] = ans / (x + y + z);
}

void solve() {
    cin >> n;
    vector<int> a(n);
    memset(dp, -1, sizeof(dp));
    vector<int> freq(4, 0);

    for (int i = 0; i < n; ++i) {
        cin >> a[i];
        freq[a[i]]++;
    }

    cout << fixed << setprecision(10) << dfs
         (freq[1], freq[2], freq[3]) << "\n";
}
```

3.8 candies

```
/*
組合 DP - O(NK)
有 N 個小孩，編號為 1,2,…,N。

他們決定將 K 顆糖果分給自己。對於每個 i (1≤i
    ≤N)，第 i 個小孩最多可以拿到 ai 顆糖果
    (包含 0 顆)。所有糖果都必須分完，不能
    剩下。

請問有多少種分配糖果的方法？請將答案對
    10^9+7 取模。若存在某個小孩分到的糖果數
    不同，則視為不同的分配方式。

令 dp[i][j] 表示將 j 顆糖果分給前 i 個小孩的
    方法數。
*/

void solve() {
    int n, k;
    cin >> n >> k;
    vector<int> a(n);
    vector<int> dp(k + 1, 0), S(k + 1, 0);

    for (int i = 0; i < n; ++i) {
        cin >> a[i];
    }

    dp[0] = 1;
    for (int i = 0; i < n; ++i) {
        vector<int> new_dp(k + 1, 0);
        S[0] = dp[0];
        for (int j = 1; j <= k; ++j) {
            S[j] = (S[j - 1] + dp[j]) % MOD;
        }
        for (int j = 0; j <= k; ++j) {
            if (j - a[i] - 1 >= 0) {
                new_dp[j] = (S[j] - S[j - a[
                    i] - 1] + MOD) % MOD;
            }
            else {
                new_dp[j ] = S[j] % MOD;
            }
        }
        dp = new_dp;
    }
    cout << dp[k] << "\n";
}
```

3.9 permutation

```
/*
抽象 DP - O(N^2)
設 N 為正整數。給定一個長度為 N-1 的字串 s，
    字元僅包含 '<' 與 '>'。

求滿足條件的排列 (p1, p2, …, pN)（即 1 到 N
    的排列）數量，答案對 10^9+7 取模：

對於每個 i（1 ≤ i ≤ N-1），若 s 的第 i 個字
    元為 '<'，則要求 p_i < p_{i+1}；若為 '>'
    ，則要求 p_i > p_{i+1}。

令 dp[i][j] 表示：在考慮前 i 個比較符號（即
    構成長度為 i+1 的排列）且最後一個元素為
    j 的有效排列數量。
*/

void solve() {
    int n;
    string s;
    cin >> n >> s;
    vector<vector<int>> dp(n + 1, vector<int
        >(n + 1, 0));
    vector<int> prefix(n + 1, 0);
    dp[1][0] = 1;

    for (int i = 2; i <= n; ++i) {
        for (int k = 0; k < n; ++k) {
            prefix[k + 1] = prefix[k] + dp[i
                - 1][k];
        }
        for (int j = 0; j < i; ++j) {
            if (s[i - 2] == '>') {
                dp[i][j] += prefix[i - 1] -
                    prefix[j];
                dp[i][j] %= MOD;
                /*
                for (int k = j; k < i - 1;
                    ++k) {
                    dp[i][j] += dp[i - 1][k
                        ];
                }
                */
            }
            else {
                dp[i][j] += prefix[j];
                dp[i][j] %= MOD;
                /*
                for (int k = 0; k < j; ++k)
                    {
                    dp[i][j] += dp[i - 1][k
                        ];
                }
                */
            }
        }
    }
    int ans = 0;
    for (int j = 0; j < n; ++j) {
        ans += dp[n][j];
        ans %= MOD;
    }
    cout << ans << "\n";
}
```

3.10 independent set

```
/*
DP on Trees - O(N)
有一棵含 N 個頂點的樹，頂點編號為 1,2,…,N。
    對於每個 i（1 ≤ i ≤ N-1），第 i 條邊連接
    頂點 x_i 和 y_i。

A 決定將每個頂點塗成白色或黑色，但不允許兩個
    相鄰的頂點同時為黑色。

求將頂點塗色的方案數，對 10^9+7 取模。

設 dp[i][j] 表示以節點 i 為根的子樹中，在節
    點 i 顏色為 j 時的塗色方案數（例如 j=0
    表示白色，j=1 表示黑色）。
*/

const int maxn = 100001;
vector<int> adj[maxn];
int f[maxn][2];
const int MOD = 1e9 + 7;

void dp(int u, int p) {

    for (int v : adj[u]) {
        if (v != p) {
            dp(v, u);
            f[u][0] = (f[v][0] + f[v][1]) %
                MOD * f[u][0] % MOD;
            f[u][1] = f[v][0] * f[u][1] %
                MOD;
        }
    }
}

void solve() {
    int n;
    cin >> n;

    for (int i = 0; i < n; ++i) {
        f[i][0] = f[i][1] = 1;
    }

    for (int i = 0; i < n - 1; ++i) {
        int u, v;
        cin >> u >> v;
        u--, v--;
        adj[u].pb(v);
        adj[v].pb(u);
    }

    dp(0, -1);

    cout << (f[0][0] + f[0][1]) % MOD << "\n
        ";
}
```

3.11 couting tower

```
/*
狀態機 DP - O(N)
你的任務是建造一座寬度為 2、高度為 n 的塔。
    你有無限數量寬度與高度為整數的方塊。

dp[i][0] = 高度為 i 的塔中，頂層為一個寬度為
    2 的方塊（即該層由跨越兩欄的單一方塊覆
    蓋）的塔的數量。
dp[i][1] = 高度為 i 的塔中，頂層在該層有兩個
    寬度為 1 的方塊（每欄各一個）的塔的數
    量。
*/

void solve() {
    long long n;
    cin >> n;
    dp[1][0] = 1;
    dp[1][1] = 1;
    for(int i = 2;i<=n;i++){
        dp[i][0] = (4 * dp[i-1][0] + dp[i
            -1][1]) % mod;
        dp[i][1] = (dp[i-1][0] + 2 * dp[i
            -1][1]) % mod;
    }
    cout << (dp[n][0] + dp[n][1]) % mod <<
        endl;
}
```

3.12 couting numbers

```
/*
區間 DP - O(digits*10)
您的任務是計算在區間 a 到 b 之間，沒有任何相
    鄰兩位數字相同的整數的個數。
```

```
定義 dp[pos][prev_digit][is_tight][
    is_started] 為從位置 pos 到結尾的有效整
    數數量，
其中 prev_digit 是位置 pos-1 的數字，
is_tight 表示是否受原數字前綴的限制，
is_started 表示是否已開始構成有效數字（用以
    避免計入前導零）。
*/

// dp[pos][prev_digit][is_tight][is_started]
int dp[20][10][2][2];
string s;

int f(int pos, int prev_digit, bool is_tight
    , bool is_started) {
    if (pos == (int)s.size()) {
        return 1;
    }
    if (dp[pos][prev_digit][is_tight][
        is_started] != -1) {
        return dp[pos][prev_digit][is_tight
            ][is_started];
    }

    int ans = 0;
    int max_d = is_tight ? (s[pos] - '0') :
        9;
    for (int d = 0; d <= max_d; ++d) {
        if (is_started && d == prev_digit) {
            continue;
        }

        bool new_is_started = is_started ||
            (d > 0);
        bool new_is_tight = is_tight && (d
            == max_d);
        ans += f(pos + 1, d, new_is_tight,
            new_is_started);
    }

    return dp[pos][prev_digit][is_tight][
        is_started] = ans;
}

int count(int x) {
    if (x < 0) return 0;
    s = to_string(x);
    memset(dp, -1, sizeof(dp));
    return f(0, 0, true, false);
}

void solve() {
    int a, b;
    cin >> a >> b;

    int ca = count(a - 1);
    int cb = count(b);
    cout << cb - ca << "\n";
}
```

4 Data Structure

4.1 undo disjoint set

```
struct DisjointSet {
  // save() is like recursive
  // undo() is like return
  int n, fa[MXN], sz[MXN];
  vector<pair<int*,int>> h;
  vector<int> sp;
  void init(int tn) {
    n=tn;
    for (int i=0; i<n; i++) sz[fa[i]=i]=1;
    sp.clear(); h.clear();
  }
  void assign(int *k, int v) {
    h.PB({k, *k});
    *k=v;
  }
  void save() { sp.PB(SZ(h)); }
  void undo() {
    assert(!sp.empty());
    int last=sp.back(); sp.pop_back();
    while (SZ(h)!=last) {
      auto x=h.back(); h.pop_back();
      *x.F=x.S;
    }
  }
  int f(int x) {
    while (fa[x]!=x) x=fa[x];
    return x;
  }
  void uni(int x, int y) {
    x=f(x); y=f(y);
    if (x==y) return ;
    if (sz[x]<sz[y]) swap(x, y);
    assign(&sz[x], sz[x]+sz[y]);
    assign(&fa[y], x);
  }
}djs;
```

4.2 lazy propagation

```
// Lazy Propagation Segment Tree supporting
    range add, range set, and range sum
    queries

int n;
int sz;
struct Node {
    int sum = 0;
    int lazy_add = 0;
    int lazy_set = 0;
    bool has_set = false;
};
vector<Node> tree;

void build(const vector<int>& a) {
    tree.resize(sz << 1);
    for (int i = 0; i < sz; ++i) {
        tree[i + sz].sum = i < n ? a[i] : 0;
    }
    for (int i = sz - 1; i >= 1; --i) {
        tree[i].sum = tree[i << 1].sum +
            tree[i << 1 | 1].sum;
    }
}

void apply_set(int idx, int l, int r, int
    val) {
    tree[idx].sum = (r - l + 1) * val;
    tree[idx].lazy_set = val;
    tree[idx].lazy_add = 0;
    tree[idx].has_set = true;
}

void apply_add(int idx, int l, int r, int
    val) {
    if (tree[idx].has_set) {
        tree[idx].lazy_set += val;
    }
    else {
        tree[idx].lazy_add += val;
    }
    tree[idx].sum += (r - l + 1) * val;
}

void push(int idx, int l, int r) {
    if (l == r) return;

    int mid = l + (r - l) / 2;
    int left = idx << 1, right = idx << 1 |
        1;

    if (tree[idx].has_set) {
        // propagate the set operation
        apply_set(left, l, mid, tree[idx].
            lazy_set);
        apply_set(right, mid + 1, r, tree[
            idx].lazy_set);
        tree[idx].has_set = false;
        tree[idx].lazy_set = 0;
        tree[idx].lazy_add = 0;
    }
    else if (tree[idx].lazy_add != 0) {
        // propagate the add operation
        apply_add(left, l, mid, tree[idx].
            lazy_add);
        apply_add(right, mid + 1, r, tree[
            idx].lazy_add);
        tree[idx].lazy_add = 0;
    }
}

void range_set(int idx, int l, int r, int ql
    , int qr, int val) {
    if (qr < l || ql > r) return;
    if (ql <= l && r <= qr) {
        apply_set(idx, l, r, val);
        return;
    }

    push(idx, l, r);
    int mid = (l + r) / 2;
    range_set(idx << 1, l, mid, ql, qr, val)
        ;
    range_set(idx << 1 | 1, mid + 1, r, ql,
        qr, val);
    tree[idx].sum = tree[idx << 1].sum +
        tree[idx << 1 | 1].sum;
}

void range_add(int idx, int l, int r, int ql
    , int qr, int val) {
    if (ql > r || qr < l) return;
    if (ql <= l && r <= qr) {
        apply_add(idx, l, r, val);
        return;
    }

    push(idx, l, r);
    int mid = (l + r) / 2;
    range_add(idx << 1, l, mid, ql, qr, val)
        ;
    range_add(idx << 1 | 1, mid + 1, r, ql,
        qr, val);
    tree[idx].sum = tree[idx << 1].sum +
        tree[idx << 1 | 1].sum;
}

int range_sum(int idx, int l, int r, int ql,
    int qr) {
    if (qr < l || ql > r) return 0;
    if (ql <= l && r <= qr) return tree[idx
        ].sum;

    push(idx, l, r);
    int mid = (l + r) / 2;
    return range_sum(idx << 1, l, mid, ql,
        qr) +
        range_sum(idx << 1 | 1, mid + 1,
            r, ql, qr);
}

void solve() {
    int q;
    cin >> n >> q;
    vector<int> a(n);

    sz = 1;
    while (sz < n) sz <<= 1;
    for (int i = 0; i < n; ++i) {
        cin >> a[i];
    }

    build(a);

    while (q--) {
        int mode, l, r, x;
        cin >> mode >> l >> r;
        l--, r--;
        if (mode == 1) {
            cin >> x;
            range_add(1, 0, sz - 1, l, r, x)
                ;
        }
        else if (mode == 2) {
            cin >> x;
            range_set(1, 0, sz - 1, l, r, x)
                ;
        }
        else {
```

```
            cout << range_sum(1, 0, sz - 1,
                 l, r) << "\n";
        }
    }
}
```

4.3 Lazy heap

```
template<typename T, typename Compare = less
    <T>>
struct lp {
    priority_queue<T, vector<T>, Compare> pq
        ;
    unordered_map<T, int> cnt;
    size_t sz = 0;
    void real_remove() {
        while(!pq.empty() && cnt[pq.top()] >
             0) {
            cnt[pq.top()]--;
            pq.pop();
        }
    }
    size_t size() {
        return sz;
    }
    void remove(T n) {
        cnt[n]++;
        sz--;
    }
    T top() {
        real_remove();
        return pq.top();
    }
    void push(T n) {
        pq.push(n);
        sz++;
    }
    void pop() {
        real_remove();
        sz--;
        pq.pop();
    }
};
```

4.4 Fenwick tree (BIT)

```
struct Fenwick {
    int n;
    vector<int> bit, prefix;
    Fenwick(int _n) : n(_n), bit(n + 1, 0),
         prefix(n + 1, 0) {}
    void update(int i, int delta) {
        for (; i <= n; i += i & -i) {
            bit[i] += delta;
        }
    }
    int query(int i) {
        int ans = 0;
        for (; i >= 1; i -= i & -i) {
            ans += bit[i];
        }
        return ans;
    }
    int range(int l, int r) {
        return query(r) - query(l - 1);
    }
};

int main() {
    int n = 5;
    vector<int> a = {0, 1, 2, 3, 4, 5};  //
        1-indexed
    Fenwick ft(n);
    for (int i = 1; i <= n; ++i) ft.update(i
        , a[i]);

    cout << ft.query(3) << "\n";    // 6
    cout << ft.range(2, 4) << "\n"; // 9

    ft.update(3, 10);                    //
        a[3] += 10
    cout << ft.query(3) << "\n";    // 16
    cout << ft.range(1, 5) << "\n"; // 25
}
```

4.5 BIT - range update + range prefix sum

```
int n, q;

void solve() {
    cin >> n >> q;
    Fenwick bit_values(n), bit_count(n);

    while (q--) {
        int mode, l, r, val;
        cin >> mode;
        if (mode == 0) {
            cin >> l >> r >> val;
            bit_values.update(l, val);
            bit_count.update(l, val * (l -
                1));
            bit_values.update(r + 1, -val);
            bit_count.update(r + 1, -val * r
                );
        }
        else {
            cin >> l >> r;
            int pref_l = bit_values.query(l
                - 1) * (l - 1) - bit_count.
                query(l - 1);
            int pref_r = bit_values.query(r)
                 * r - bit_count.query(r);
            cout << pref_r - pref_l << "\n";
        }
    }
}
```

4.6 Order Statistics Tree

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

template <class T>
using ordered_set =
    tree<T, null_type, less<T>, rb_tree_tag,
         tree_order_statistics_node_update>;

int main() {
  int n;
  cin >> n;
  ordered_set<int> st;
  vector<int> p(n);
  for (int i = 0; i < n; i++) {
    st.insert(i);
    cin >> p[i];
  }
  for (int i = 0; i < n; i++) {
    int ind;
    cin >> ind;
    ind--;
    int pos = *st.find_by_order(ind);
    cout << st.order_of_key(pos) << ' ';
    st.erase(pos);
    cout << p[pos] << (i == n - 1 ? '\n' : '
        ');
  }
}
```

4.7 Trie (Prefix tree)

```
struct trie {
    int n = 0;
    trie *a[2];
    trie() {
        a[0] = a[1] = nullptr;
    }

    void insert(int k) {
        trie *curr = this;
        for (int i = 63; i >= 0; --i) {
            bool bit = (k & (1LL << i)) > 0;
            if (curr->a[bit] == nullptr) {
                curr->a[bit] = new trie();
            }
            curr = curr->a[bit];
            curr->n++ ;
        }
    }

    void erase(int k) {
        trie *curr = this;
        for (int i = 63; i >= 0; --i) {
            bool bit = (k & (1LL << i)) > 0;
            curr = curr->a[bit];
            curr->n--;
        }
    }

    int query(int k) {
        trie *curr = this;
        int x = 0;
        for (int i = 63; i >= 0; --i) {
            x ^= (1LL << i) & k;
            x ^= 1LL << i;
            bool bit = (k & (1LL << i)) ==
                0;
            if (curr->a[bit] == nullptr ||
                curr->a[bit]->n == 0)
                x ^= 1LL << i, bit ^= 1;
            curr = curr->a[bit];
        }
        return x;
    }
};
```

4.8 segment tree

```
int sz = 1;
const int maxn = 2e6;
int t[maxn << 1];

void build(const vector<int>& a) {
    for (int i = 0; i < n; ++i) {
        t[i + sz] = a[i];
    }
    for (int i = sz - 1; i >= 1; --i) {
        t[i] = max(t[i << 1], t[i << 1 | 1])
            ;
    }
}

void update(int i, int val) {
    i += sz;
    t[i] = val;
    for (i >>= 1; i >= 1; i >>= 1) {
        t[i] = max(t[i << 1], t[i << 1 | 1])
            ;
    }
}

int query(int r) {
    if (t[1] < r) return 0;

    int i = 1;
    while (i < sz) {
        if (t[i << 1] >= r) i = i << 1;
        else i = i << 1 | 1;
    }
    return i - sz + 1;
}
```

4.9 BIT range update point query

```
// Fenwick Tree (Binary Indexed Tree) for
    Range Updates and Point Queries

template<typename T>
class BIT {
#define ALL(x) begin(x), end(x)
private:
    vector<T> arr;
```

```
    int n;
    inline int lowbit(int x) { return x & (-x); }
    void addInternal(int s, T v) {
        while (s > 0) {
            arr[s] += v;
            s -= lowbit(s);
        }
    }
public:
    void init(int n_) {
        n = n_;
        arr.resize(n + 1);
        fill(ALL(arr), 0);
    }
    void add(int l, int r, T v) {
        // add v to interval (l, r], 1-based
        addInternal(l, -v);
        addInternal(r, v);
    }
    T query(int x) {
        // value at index x
        T res = 0;
        while (x <= n) {
            res += arr[x];
            x += lowbit(x);
        }
        return res;
    }
#undef ALL
};

int main() {
    BIT<int> bit;
    bit.init(5);

    // add +3 to indices 1..3
    bit.add(0, 3, 3);

    // add +2 to indices 3..5
    bit.add(2, 5, 2);

    cout << "Value at 1 = " << bit.query(1) << "\n"; // expect 3
    cout << "Value at 3 = " << bit.query(3) << "\n"; // expect 3+2=5
    cout << "Value at 5 = " << bit.query(5) << "\n"; // expect 2
}
```

4.10 DSU

```
// fast disjoint set union
class DSU {
    vector<int> pa, sz;
public:
    explicit DSU(int n) : pa(n, -1), sz(n, 1) {}
    int find(int x) { // collapsing find
        return pa[x] == -1 ? x : pa[x] = find(pa[x]);
    }
    void unite(int x, int y) { // weighted union
        auto rx{find(x)}, ry{find(y)};
        if (rx == ry) return ;
        if (sz[rx] < sz[ry]) swap(rx, ry);
        pa[ry] = rx, sz[rx] += sz[ry];
    }
};
```

5 Geometry

5.1 cross. product

```
// If cross(a,b,c) > 0, c is to the left of line ab
// If cross(a,b,c) < 0, c is to the right of line ab
// If cross(a,b,c) = 0, a, b, c are collinear

// point struct version
// 2D Point structure
struct Point {
    double x, y;
};

// Cross product (scalar in 2D)
double cross(const Point& a, const Point& b, const Point& c) {
    // Computes (b - a) × (c - a)
    return (b.x - a.x) * (c.y - a.y) -
           (b.y - a.y) * (c.x - a.x);
}

// std::complex version
typedef std::complex<double> point;
#define x real()
#define y imag()

double cross(const point &a, const point &b) {
    return (std::conj(a) * b).imag();
}
```

5.2 Check Point in Polygon

```
/*
We can cast a ray from the point P going in any fixed direction (people usually go to the right).
If the point is located on the outside of the polygon the ray will intersect its edges an even number of times.
If the point is on the inside of the polygon then it will intersect the edge an odd number of times.
*/

struct Point {
    int x, y;
};

int cross(const Point& a, const Point& b, const Point& c) {
    // return (c - a) x (c - b)
    int ans = (c.x - a.x) * (c.y - b.y) - (c.x - b.x) * (c.y - a.y);
    if (ans == 0) return 0;
    return ans > 0 ? 1 : -1;
}

bool intersect(const Point& a, const Point& b, const Point& c, const Point& d) {
    int c1 = cross(a, b, c);
    int c2 = cross(a, b, d);
    int c3 = cross(c, d, a);
    int c4 = cross(c, d, b);

    return (c1 * c2 < 0 && c3 * c4 < 0);
}

bool onSegment(const Point& a, const Point& b, const Point& c) {
    return (cross(a, b, c) == 0 &&
            min(a.x, b.x) <= c.x && c.x <= max(a.x, b.x) &&
            min(a.y, b.y) <= c.y && c.y <= max(a.y, b.y));
}

void solve() {
    int n, m;
    cin >> n >> m;
    vector<Point> poly(n);

    for (int i = 0; i < n; ++i) {
        int a, b;
        cin >> a >> b;
        poly[i] = {a, b};
    }

    // set a point at outside point to form the ray
    Point outside = {(int)2e9 + 7, (int)2e9 + 13};
    for (int i = 0; i < m; ++i) {
        int a, b;
        bool found = false;
        cin >> a >> b;
        Point p = {a, b};

        int cnt = 0;
        if (intersect(poly.back(), poly[0], p, outside)) {
            cnt++;
        }
        if (onSegment(poly.back(), poly[0], p)) {
            cout << "BOUNDARY\n";
            found = true;
        }
        for (int j = 0; j < n - 1 && !found; ++j) {
            if (intersect(poly[j], poly[j + 1], p, outside)) {
                cnt++;
            }
            if (onSegment(poly[j], poly[j + 1], p)) {
                cout << "BOUNDARY\n";
                found = true;
            }
        }
        if (found) continue;
        if (cnt & 1) {
            cout << "INSIDE\n";
        }
        else {
            cout << "OUTSIDE\n";
        }
    }
}
```

5.3 Point

```
#pragma once

template <class T> int sgn(T x) { return (x > 0) - (x < 0); }
template<class T>
struct Point {
  typedef Point P;
  T x, y;
  explicit Point(T x=0, T y=0) : x(x), y(y) {}
  bool operator<(P p) const { return tie(x,y) < tie(p.x,p.y); }
  bool operator==(P p) const { return tie(x,y)==tie(p.x,p.y); }
  P operator+(P p) const { return P(x+p.x, y+p.y); }
  P operator-(P p) const { return P(x-p.x, y-p.y); }
  P operator*(T d) const { return P(x*d, y*d); }
  P operator/(T d) const { return P(x/d, y/d); }
  T dot(P p) const { return x*p.x + y*p.y; }
  T cross(P p) const { return x*p.y - y*p.x; }
  T cross(P a, P b) const { return (a-*this).cross(b-*this); }
  T dist2() const { return x*x + y*y; }
  double dist() const { return sqrt((double)dist2()); }
  // angle to x-axis in interval [-pi, pi]
  double angle() const { return atan2(y, x); }
  P unit() const { return *this/dist(); } // makes dist()=1
  P perp() const { return P(-y, x); } // rotates +90 degrees
  P normal() const { return perp().unit(); }
  // returns point rotated 'a' radians ccw around the origin
  P rotate(double a) const {
```

```
    return P(x*cos(a)-y*sin(a),x*sin(a)+y*
        cos(a)); }
  friend ostream& operator<<(ostream& os, P
      p) {
    return os << "(" << p.x << "," << p.y <<
        ")"; }
};
```

5.4 Polygon Lattice Points

```
/*
Pick's Theorem:
For a simple polygon with integer
    coordinates, the area A, the number of
    interior lattice points a
nd the number of boundary lattice points b
    satisfy:
A = a + b/2 - 1
*/

/*
The number of intersections of a line with
    lattice points is the greatest common
    divisor of
P1.x - P2.x and P1.y - P2.y
*/
int countOnSegment(const Point& a, const
    Point& b) {
    int dx = abs(a.x - b.x);
    int dy = abs(a.y - b.y);
    return __gcd(dx, dy);
}

void solve() {
    int n;
    cin >> n;
    vector<Point> poly(n);

    for (int i = 0; i < n; ++i) {
        cin >> poly[i].x >> poly[i].y;
    }

    // b: boundary points
    int b = countOnSegment(poly.back(), poly
        [0]);
    for (int i = 0; i < n - 1; ++i) {
        b += countOnSegment(poly[i], poly[i
            + 1]);
    }
    int area2 = abs(polygonArea2(poly));
    // a: interior points
    int a = (area2 + 2 - b) / 2;
    cout << a << " " << b << "\n";
}
```

5.5 line intersect

```
// 2D Point structure
struct Point {
    double x, y;
};

// Cross product (scalar in 2D)
double cross(const Point& a, const Point& b,
     const Point& c) {
    // Computes (b - a) × (c - a)
    return (b.x - a.x) * (c.y - a.y) -
           (b.y - a.y) * (c.x - a.x);
}

// Check if value is between two others (
    with endpoints allowed)
bool between(double a, double b, double x) {
    return min(a, b) <= x && x <= max(a, b);
}

// Check if point c lies on segment ab
bool onSegment(const Point& a, const Point&
    b, const Point& c) {
    return cross(a, b, c) == 0 &&
           between(a.x, b.x, c.x) &&
           between(a.y, b.y, c.y);
}

// Main intersection check
bool segmentsIntersect(const Point& A, const
     Point& B,
                       const Point& C, const
                           Point& D) {
    double c1 = cross(A, B, C);
    double c2 = cross(A, B, D);
    double c3 = cross(C, D, A);
    double c4 = cross(C, D, B);

    // General case
    if (c1 * c2 < 0 && c3 * c4 < 0) return
        true;

    // Special cases: collinear +
        overlapping
    if (c1 == 0 && onSegment(A, B, C))
        return true;
    if (c2 == 0 && onSegment(A, B, D))
        return true;
    if (c3 == 0 && onSegment(C, D, A))
        return true;
    if (c4 == 0 && onSegment(C, D, B))
        return true;

    return false;
}

int main() {
    Point A{0, 0}, B{4, 4};
    Point C{0, 4}, D{4, 0};

    if (segmentsIntersect(A, B, C, D))
        cout << "Segments intersect\n";
    else
        cout << "Segments do not intersect\n
            ";

    return 0;
}
```

5.6 Polygon area

```
// Returns twice the signed area of a
    polygon.
#pragma once

#include "Point.h"

template<class T>
T polygonArea2(vector<Point<T>>& v) {
  T a = v.back().cross(v[0]);
    for (int i = 0; i < (int)v.size() - 1;
        ++i) {
        a += v[i].cross(v[i+1]);
    }
  return a;
}

int main() {
    // Example: square with vertices (0,0),
        (0,1), (1,1), (1,0)
    vector<Point<long long>> poly = {
        {0,0}, {0,1}, {1,1}, {1,0}
    };

    long long area2 = polygonArea2(poly);
    cout << "Twice signed area = " << area2
        << "\n";
    cout << "Absolute area = " << abs(area2)
        / 2.0 << "\n";

    return 0;
}
```

5.7 using std::complex

```
#include <iostream>
#include <complex>
#include <cmath>

typedef std::complex<double> point;
#define x real()
#define y imag()

constexpr double PI = std::acos(-1.0);
// conj(a) = a 的共軛 · reflection of a
    across x-axis

// Basic operations
double dot(const point &a, const point &b) {
     return (std::conj(a) * b).real(); }
double cross(const point &a, const point &b)
     { return (std::conj(a) * b).imag(); }

// Projections / reflections / geometry
    helpers
point project_onto_vector(const point &p,
    const point &v) {
    return v * (dot(p, v) / std::norm(v));
}

point project_onto_line(const point &p,
    const point &a, const point &b) {
    return a + (b - a) * (dot(p - a, b - a)
        / std::norm(b - a));
}

point reflect_across_line(const point &p,
    const point &a, const point &b) {
    return a + std::conj((p - a) / (b - a))
        * (b - a);
}

point intersection(const point &a, const
    point &b, const point &p, const point &q
    ) {
    double c1 = cross(p - a, b - a), c2 =
        cross(q - a, b - a);
    return (c1 * q - c2 * p) / (c1 - c2); //
        undefined if parallel (divide by
        zero)
}

double angle_between(const point &a, const
    point &b) {
    // angle of vector b - a
    return std::arg(b - a);
}

double angle_ABC(const point &a, const point
    &b, const point &c) {
    double r = std::remainder(std::arg(a - b
        ) - std::arg(c - b), 2.0 * PI);
    return std::abs(r);
}

int main() {
    // sample
    point a = 2.0;            // (2,0)
    point b(3.0, 7.0);        // (3,7)
    std::cout << a << ' ' << b << '\n'; //
        (2,0) (3,7)
    std::cout << a + b << '\n';         //
        (5,7)

    // usage examples
    point p1(3, 2), p2(2, -7);
    std::cout << "p1 + p2 = " << p1 + p2 <<
        '\n';   // (5,-5)
    std::cout << "p1 - p2 = " << p1 - p2 <<
        '\n';   // (1,9)
    std::cout << "3.0 * p1 = " << 3.0 * p1
        << '\n'; // (9,6)
    std::cout << "p1 / 5.0 = " << p1 / 5.0
        << '\n'; // (0.6,0.4)

    // dot / cross via complex
    std::cout << "dot(p1,p2) = " << dot(p1,
        p2) << '\n';
    std::cout << "cross(p1,p2) = " << cross(
        p1, p2) << '\n';

    // distances, angle, rotation, polar/
        cartesian
    std::cout << "squared dist = " << std::
        norm(p1 - p2) << '\n';
```

```
    std::cout << "euclid dist = " << std::
        abs(p1 - p2) << '\n';
    std::cout << "angle p1->p2 = " <<
        angle_between(p1, p2) << '\n';
    std::cout << "angle ABC = " << angle_ABC
        (point(1,0), point(0,0), point(0,1))
         << '\n';

    // project / reflect / intersect
        examples
    point v(1, 1);
    point proj = project_onto_vector(p1, v);
    std::cout << "project p1 onto v = " <<
        proj << '\n';

    point lineA(0,0), lineB(2,0);
    std::cout << "project p1 onto line = "
        << project_onto_line(p1, lineA,
        lineB) << '\n';
    std::cout << "reflect p1 across line = "
         << reflect_across_line(p1, lineA,
        lineB) << '\n';

    // intersection example
    point r1a(0,0), r1b(1,1);
    point r2a(0,1), r2b(1,0);
    std::cout << "intersection = " <<
        intersection(r1a, r1b, r2a, r2b) <<
        '\n';

    // polar/cartesian and rotation
    point polar_pt = std::polar(5.0, PI/4);
    std::cout << "polar(5,PI/4) = " <<
        polar_pt << '\n';
    point rotated = p1 * std::polar(1.0, PI
        /2); // rotate p1 by 90 degrees
        about origin
    std::cout << "rotate p1 by 90deg = " <<
        rotated << '\n';

    return 0;
}
```

5.8 point distance from a line

```
// calculate area using cross product and
     divide by base length
double point_line_distance(const point &p,
     const point &a, const point &b) {
    // area = 0.5 * |cross(b - a, p - a)|
    // base = |b - a|
    // height = 2 * area / base = |cross(b -
         a, p - a)| / |b - a|
    std::abs(cross(b - a, p - a)) / std::abs
        (b - a);
}
```

6 Graph

6.1 Prim

```
// Prim's algorithm
vector<tuple<int, int, long long>> Prim(
     const vector<vector<pair<int, long long
     >>>& adj) {
    const auto& n = adj.size();
    vector<tuple<int, int, long long>> mst
        {};

    vector<bool> found(n, false);
    using ti = tuple<long long, int, int>;
    priority_queue<ti, vector<ti>, greater<
        ti>> pq{};
    found[0] = true;
    for (auto& [v, w] : adj[0]) pq.emplace(w
        , 0, v);

    for (int i = 0; i < n - 1; ++i) {
        int mn, u, v;
        do {
            tie(mn, u, v) = pq.top(), pq.pop
                ();
        } while (found[v]);
        found[v] = true, mst.emplace_back(u,
             v, mn);
        for (auto& [x, w] : adj[v]) pq.
            emplace(w, v, x);
    }
    return mst;
}
```

6.2 Eulerian cycle

```
// Eulerian cycle in an undirected graph

vector<int> euler_cycle(vector<vector<pair<
    int, int>>>& adj, int w = 0) {
    int n{adj.size()}, m{};
    for (int v{0}; v < n; ++v) m += adj[v].
        size();
    m /= 2;

    vector<int> res{};
    stack<pair<int, int>> stk{};
    stk.emplace(w, -1);
    vector<int> nxt(n);
    vector<bool> usd(m);
    while (!stk.empty()) {
        auto [u, i]{stk.top()};
        while (nxt[u] < adj[u].size() && usd
            [adj[u][nxt[u]].second]) ++nxt[u
            ];
        if (nxt[u] < adj[u].size()) {
            auto [v, j]{adj[u][nxt[u]]};
            ++nxt[u], usd[j] = true, stk.
                emplace(v, j);
        } else {
            if (i != -1) res.push_back(i);
            stk.pop();
        }
    }
    return res;
}

int main() {
    int n = 4; // number of vertices
    vector<vector<pair<int, int>>> adj(n);

    // Add edges with edge IDs
    int eid = 0;
    auto add_edge = [&](int u, int v) {
        adj[u].push_back({v, eid});
        adj[v].push_back({u, eid});
        ++eid;
    };

    add_edge(0, 1);
    add_edge(1, 2);
    add_edge(2, 3);
    add_edge(3, 0);

    vector<int> cycle = euler_cycle(adj, 0);

    cout << "Euler cycle (edge IDs in order)
        : ";
    for (int id : cycle) cout << id << " ";
    cout << "\n";

    return 0;
}
```

6.3 maximum weight bipartite matching

```
// maximum weight matching in a weighted
     bipartite graph
// Hungarian Algorithm - O(V^3)
class bipartiteGraph {
    int n;
    vector<long long> lx, ly;
    vector<bool> vx, vy;
    queue<int> qu{}; // only X's vertices
    vector<int> py;
    vector<long long> dy; // smallest (lx[x]
         + ly[y] - w[x][y])
    vector<int> pdy; // & which x
    void relax(int x) {
        for (int y{0}; y < n; ++y)
            if (lx[x] + ly[y] - w[x][y] < dy
                [y])
                dy[y] = lx[x] + ly[y] - w[x
                    ][y], pdy[y] = x;
    }
    void augment(int y) {
        while (y != -1) {
            int x{py[y]}, yy{mx[x]};
            mx[x] = y, my[y] = x;
            y = yy;
        }
    }
    bool bfs() {
        while (!qu.empty()) {
            int x{qu.front()}; qu.pop();
            for (int y{0}; y < n; ++y) {
                if (!vy[y] && lx[x] + ly[y]
                    == w[x][y]) {
                    vy[y] = true, py[y] = x;
                    if (my[y] == -1) return
                        augment(y), true;
                    int xx{my[y]};
                    qu.push(x), vx[xx] =
                        true, relax(xx);
                }
            }
        }
        return false;
    }
    void relabel() {
        long long d{numeric_limits<long long
            >::max()};
        for (int y{0}; y < n; ++y) if (!vy[y
            ]) d = min(d, dy[y]);
        for (int x{0}; x < n; ++x) if (vx[x
            ]) lx[x] -= d;
        for (int y{0}; y < n; ++y) if (vy[y
            ]) ly[y] += d;
        for (int y{0}; y < n; ++y) if (!vy[y
            ]) dy[y] -= d;
    }
    bool expand() { // expand one layer with
         new equality edges
        for (int y{0}; y < n; ++y) {
            if (!vy[y] && dy[y] == 0) {
                vy[y] = true, py[y] = pdy[y
                    ];
                if (my[y] == -1) return
                    augment(y), true;
                int xx{my[y]};
                qu.push(xx), vx[xx] = true,
                    relax(xx);
            }
        }
        return false;
    }
public:
    vector<vector<long long>> w;
    vector<int> mx, my;
    bipartiteGraph(int _n) : n{_n}, lx(n),
        ly(n), vx(n), vy(n), py(n), dy(n),
        pdy(n),
        w(n, vector<long long>(n, 0)), mx(n,
            -1), my(n, -1) {}
    long long Hungarian() {
        for (int i{0}; i < n; ++i) {
            lx[i] = ly[i] = 0;
            for (int j{0}; j < n; ++j)
                lx[i] = max(lx[i], w[i][j]);
            mx[i] = my[i] = -1;
        }

        for (int x{0}; x < n; ++x) {
            fill(vx.begin(), vx.end(), false
                );
            fill(vy.begin(), vy.end(), false
                );
```

```
            fill(dy.begin(), dy.end(),
                 numeric_limits<long long>::
                 max());

            qu = {}, qu.push(x), vx[x] =
                 true, relax(x);
            while (!bfs()) {
                relabel();
                if (expand()) break;
            }
        }

        long long weight{0};
        for (int x{0}; x < n; ++x) weight +=
             w[x][mx[x]];
        return weight;
    }
};

// usage example:
int main() {
    int n = 4;
    bipartiteGraph g(n);
    long long mat[4][4] = {
        {9, 2, 7, 8},
        {6, 4, 3, 7},
        {5, 8, 1, 8},
        {7, 6, 9, 4}
    };
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            g.w[i][j] = mat[i][j];

    long long maxWeight = g.Hungarian();
    std::cout << "Maximum weight: " <<
         maxWeight << "\n";
    for (int i = 0; i < n; ++i)
        std::cout << "X " << i << " matched
             with Y " << g.mx[i] << "\n";
}
```

6.4 maximum flow

```
// minimum cut maximum flow
// relabel-to-front algorithm
// (an implemention of generic push-relabel
     algorithm)

// handle the capacity, c, of a vertex, v.
// Add a new vertex, v', and an edge v-v'
     with capacity c.
template<typename T> //requires
     signed_integral<T>
void handle_vertex_capacity(int u, T c, int&
      n, vector<tuple<int, int, T>>& e) {
    int w{n++};
    for (auto& [_u, _v, _c] : e) if (_u == u
         ) _u = w;
    e.emplace_back(u, w, c);
}

template<typename T> //requires
     signed_integral<T>
class flowNetwork {
    int n, s, t;
    vector<int> h; // height label
    vector<T> e; // excess
    vector<vector<int>> adj; // adjacent
         list
    vector<vector<T>> c, r{};
    vector<size_t> p; // record the position
          processed
    list<int> lst{}; // topological sort of
         admissible network
    void push(int u, int v) {
        T f{min(e[u], r[u][v])};
        r[u][v] -= f, r[v][u] += f;
        e[u] -= f, e[v] += f;
    }
    void relabel(int u) {
        int mn{numeric_limits<int>::max()};
        for (auto& v : adj[u])
            if (r[u][v]) mn = min(mn, h[v]);
        h[u] = mn + 1;
    }
    void init() {
        fill(h.begin(), h.end(), 0);
        fill(e.begin(), e.end(), 0);
        lst.clear();
        for (int u{0}; u < n; ++u) if (u !=
             s && u != t) lst.push_back(u);
        fill(p.begin(), p.end(), 0);
        r = c;
    }
    void preflow() {
        h[s] = n;
        for (auto& v : adj[s]) {
            e[v] += r[s][v], e[s] -= r[s][v
                 ];
            r[v][s] += r[s][v], r[s][v] = 0;
        }
    }
    bool discharge(int u) {
        bool flag{false};
        while (e[u]) {
            for (; p[u] < adj[u].size(); ++p
                 [u]) {
                int v{adj[u][p[u]]};
                if (r[u][v] && h[u] == h[v]
                     + 1) push(u, v);
                if (!e[u]) break;
            }
            if (e[u]) relabel(u), p[u] = 0,
                 flag = true;
        }
        return flag; // return if relabel or
             not
    }
public:
    flowNetwork(int _n) : n{_n}, s{0}, t{n -
         1}, h(n), e(n), adj(n), c(n, vector
        <T>(n)), p(n) {}
    void set_st(int _s, int _t) {
        s = _s, t = _t;
    }
    void add_edge(int u, int v, T _c) {
        if (!c[u][v] && !c[v][u]) adj[u].
             push_back(v), adj[v].push_back(u
             );
        c[u][v] += _c;
    }
    T max_flow() {
        assert(s != t);

        init();
        preflow();

        auto it{lst.begin()};
        while (it != lst.end()) {
            if (discharge(*it)) { // relabel
                 -to-front
                lst.push_front(*it), lst.
                     erase(it);
                it = lst.begin();
            }
            it = next(it);
        }

        return e[t];
    };
};


int main() {
    using T = int;

    // Build an example graph with a vertex
         capacity on node 2.
    int n = 4; // nodes: 0 (source), 1, 2, 3
          (sink)
    vector<tuple<int, int, T>> edges = {
        {0, 1, 3},
        {0, 2, 2},
        {1, 2, 5},
        {1, 3, 2},
        {2, 3, 3}
    };

    // Enforce vertex capacity of 2 on node
         2.
    handle_vertex_capacity<T>(2, 2, n, edges
         );

    // Build the network and compute max
         flow.
    flowNetwork<T> net(n);
    net.set_st(0, 3);
    for (auto [u, v, cap] : edges) net.
         add_edge(u, v, cap);

    cout << "Max flow = " << net.max_flow()
         << '\n';
}
```

6.5 maximum cardinality bipartite matching

```
class BipartiteMatching {
    vector<bool> vis;
    vector<int> mx{}, my{};
    bool dfs(const vector<vector<int>>& adj,
          int x) {
        for (int y : adj[x]) {
            if (vis[y]) continue;
            vis[y] = true;
            if (my[y] == -1 || dfs(adj, my[y
                 ])) {
                mx[x] = y; my[y] = x;
                return true;
            }
        }
        return false;
    }
public:
    pair<vector<int>, vector<int>> operator
         ()(const vector<vector<int>>& adj,
         int ny) {
        vis.assign(ny, false);
        mx.assign((int)adj.size(), -1);
        my.assign(ny, -1);
        for (int x = 0; x < (int)adj.size();
             ++x) {
            fill(vis.begin(), vis.end(),
                 false);
            dfs(adj, x);
        }
        return {mx, my};
    }
};

// Usage example:
int main() {
    // Left partition has 4 nodes (0..3),
         right partition has 4 nodes (0..3).
    vector<vector<int>> adj(4);
    adj[0] = {0, 1};
    adj[1] = {1, 2};
    adj[2] = {2};
    adj[3] = {2, 3};

    BipartiteMatching bm;
    auto res = bm(adj, 4);
    const auto& mx = res.first;

    cout << "Matched pairs (left -> right):\
        n";
    for (int i = 0; i < (int)mx.size(); ++i)
        if (mx[i] != -1)
            cout << i << " -> " << mx[i] <<
                 "\n";
}
```

6.6 Floyd-Warshall

```
// Floyd-Warshall algorithm
template<typename T>
vector<vector<optional<T>>> Floyd_Warshall(
     const vector<vector<optional<T>>>& adj)
     {
    const auto& n{adj.size()};
    auto d{adj};

    for (int i{0}; i < n; ++i) d[i][i] = 0;
```

```
    for (int k{0}; k < n; ++k)
        for (int i{0}; i < n; ++i)
            for (int j{0}; j < n; ++j) {
                if (!d[i][k] || !d[k][j])
                    continue; // no value
                if (!d[i][j] || d[i][j] > d[
                    i][k].value() + d[k][j].
                    value())
                    d[i][j] = d[i][k].value
                        () + d[k][j].value()
                        ;
            }

    return d;
}
```

6.7 MST

```
// Kruskal' s algorithm
vector<tuple<int, int, long long>> Kruskal(
    int n, vector<tuple<long long, int, int
    >>& edges) {
    vector<tuple<int, int, long long>> mst
        {};
    DSU dsu{n};

    sort(edges.begin(), edges.end());
    for (auto& [w, u, v] : edges)
        if (dsu.find(u) != dsu.find(v))
            dsu.unionn(u, v), mst.
                emplace_back(u, v, w);
    return mst;
}
```

6.8 Hopcroft–Karp

```
// maximum cardinality bipartite matching in
     unweighted bipartite graph
// Hopcroft – Karp algorithm - O(EVV)
class bipartiteGraph {
    int nx, ny;
    vector<int> dx, dy;
    bool dfs(int y) {
        for (auto& x : adjy[y]) {
            if (dx[x] + 1 != dy[y]) continue
                ;
            if (mx[x] == -1 || dfs(mx[x])) {
                my[y] = x, mx[x] = y;
                dy[y] = -1; // augmented -
                    remove from BFS forest
                return true;
            }
        }
        dy[y] = -1; // no augmenting path -
            remove from BFS forest
        return false;
    }
    bool bfs() {
        fill(dx.begin(), dx.end(), -1);
        fill(dy.begin(), dy.end(), -1);
        queue<int> qu{}, qu2{};
        for (int x{0}; x < nx; ++x) // use
            all unmatched x's as roots
            if (mx[x] == -1) qu.push(x), dx[
                x] = 0;

        bool found{false};
        while (!qu.empty() && !found) { //
            stop at the level found
            while (!qu.empty()) {
                auto x{qu.front()}; qu.pop()
                    ;
                for (auto& y : adjx[x])
                    if (dy[y] == -1 && my[y]
                        != x) {
                        dy[y] = dx[x] + 1;
                        if (my[y] == -1)
                            found = true;
                        else qu2.push(my[y])
                            , dx[my[y]] = dy
                            [y] + 1;
                    }
            }
            qu.swap(qu2);
        }
        return found;
    }
    vector<vector<int>> adjx, adjy;
public:
    vector<int> mx, my;
    bipartiteGraph(int _nx, int _ny) : nx{
        _nx}, ny{_ny}, dx(nx), dy(ny)
        , adjx(nx), adjy(ny), mx(nx, -1), my
            (ny, -1) {}
    void addEdge(int x, int y) {
        adjx[x].push_back(y), adjy[y].
            push_back(x);
    }
    int Hopcroft_Karp() {
        int c{0};
        while (bfs()) {
            for (int y{0}; y < ny; ++y)
                if (my[y] == -1 && dy[y] !=
                    -1) c += dfs(y);
        }
        return c;
    }
};

// Usage example:
int main() {
    // Left partition has 4 nodes (0..3),
        right partition has 4 nodes (0..3).
    bipartiteGraph bg(4, 4);
    bg.addEdge(0, 0);
    bg.addEdge(0, 1);
    bg.addEdge(1, 1);
    bg.addEdge(1, 2);
    bg.addEdge(2, 2);
    bg.addEdge(3, 2);
    bg.addEdge(3, 3);

    int max_matching = bg.Hopcroft_Karp();
    cout << "Maximum matching size: " <<
        max_matching << "\n";
    cout << "Matched pairs (left -> right):\
        n";
    for (int i = 0; i < (int)bg.mx.size();
        ++i)
        if (bg.mx[i] != -1)
            cout << i << " -> " << bg.mx[i]
                << "\n";
}
```

6.9 Dijkstra

```
// Dijkstra algorithm
template<typename T>
vector<optional<T>> Dijkstra(const vector<
    vector<pair<int, T>>>& adj, int s) {
    const auto& n{adj.size()};
    vector<optional<T>> d(n, nullopt);
    d[s] = 0;

    vector<bool> found(n, false);
    using pq_t = pair<T, int>;
    priority_queue<pq_t, vector<pq_t>,
        greater<pq_t>> pq{};
    pq.emplace(0, s);
    while (!pq.empty()) {
        auto [_, u]{pq.top()}; pq.pop();
        if (found[u]) continue;
        found[u] = true;
        for (auto& [v, w] : adj[u])
            if (!d[v] || d[v] > d[u].value()
                + w) {
                d[v] = d[u].value() + w;
                pq.emplace(d[v].value(), v);
            }
    }

    return d;
}
```

6.10 Dinic

```
int n, s, t, cnt, inf = 1e17;
vector<vector<int>> adj;
int level[100000], ptr[100000], rev[100000],
    to[100000], cap[100000];
void add(int a, int b, int c) {
    int id = cnt++, rid = cnt++;
    to[id] = b;
    to[rid] = a;
    cap[id] = c;
    cap[rid] = 0;

    rev[id] = rid;
    rev[rid] = id;
    adj[a].push_back(id);
    adj[b].push_back(rid);
}
bool bfs() {
    memset(level, -1, sizeof(level));
    queue<int> q;
    q.push(s);
    level[s] = 0;

    while(!q.empty()) {
        int curr = q.front(); q.pop();
        for(int id: adj[curr]) {
            int next = to[id];
            if(cap[id] > 0 && level[next] ==
                -1) {
                level[next] = level[curr] +
                    1;
                q.push(next);
            }
        }
    }
    return level[t] != -1;
}
int dfs(int curr, int flow) {
    if(curr == t || flow == 0) return flow;

    for(; ptr[curr] < adj[curr].size(); ptr[
        curr]++) {
        int id = adj[curr][ptr[curr]];
        int next = to[id];

        if(cap[id] > 0 && level[next] ==
            level[curr] + 1) {
            int temp = dfs(next, min(flow,
                cap[id]));
            if(temp == 0) continue;

            int rid = rev[id];
            cap[id] -= temp;
            cap[rid] += temp;
            return temp;
        }
    }
    return 0;
}
int maxflow() {
    int flow = 0;
    while(bfs()) {
        memset(ptr, 0, sizeof(ptr));
        while(true) {
            int temp = dfs(s, inf);
            if(temp == 0) break;
            flow += temp;
        }
    }
    return flow;
}
```

6.11 maximum cardinality matching

```
// maximum matching
// Edmonds' Blossom algorithm
// O(VEa(E,V))
class graph {
    int n;
    vector<int> p, d, a, c1, c2;
    // (alternating tree), (unvisited: -1,
        even: 0, odd: 1), (auxiliary for lca
        ), (cross edge)
```

```
    int label{0};
    /* DSU */
    vector<int> dsu_p, dsu_sz, dsu_b;
    void dsu_reset() {
        fill(dsu_p.begin(), dsu_p.end(), -1);
        fill(dsu_sz.begin(), dsu_sz.end(), 1);
        iota(dsu_b.begin(), dsu_b.end(), 0);
    }
    int find(int x) { // collapsing find
        return dsu_p[x] == -1 ? x : dsu_p[x] = find(dsu_p[x]);
    }
    int base(int x) { return dsu_b[find(x)]; }
    void contract(int x, int y) { // weighted union
        auto rx{find(x)}, ry{find(y)};
        if (rx == ry) return ;
        auto b{dsu_b[rx]}; // treat x's base as new base
        if (dsu_sz[rx] < dsu_sz[ry]) swap(rx, ry);
        dsu_p[ry] = rx, dsu_sz[rx] += dsu_sz[ry], dsu_b[rx] = b;
    }
    /*******/
    queue<int> qu{}; // only even vertices
    void handle_one_side(int x, int y, int b) {
        for (int v{base(x)}; v != b; v = base(p[v])) {
            c1[v] = x, c2[v] = y, contract(b, v);
            if (d[v] == 1) qu.push(v); // odd vertex -> even vertex
        }
    }
    int lca(int u, int v) {
        ++label; // use next number in order not to clear a
        while (true) {
            if (a[u] == label) return u;
            a[u] = label;
            if (p[u] != -1) u = base(p[u]);
            swap(u, v);
        }
    }
    void augment(int r, int y) {
        if (r == y) return ;
        if (d[y] == 0) { // even vertex -> odd vertex
            // check d[y] == 0 instead of d[p[y]] == 1, so (m[y], y) is in the blossom
            augment(m[y], m[c1[y]]);
            augment(r, m[c2[y]]);
            m[c1[y]] = c2[y], m[c2[y]] = c1[y];
        } else {
            int x{p[y]};
            augment(r, m[x]);
            m[x] = y, m[y] = x;
        }
    }
    bool bfs(int r) {
        dsu_reset();
        fill(d.begin(), d.end(), -1);

        qu = {}, qu.push(r), d[r] = 0;
        while (!qu.empty()) {
            int x{qu.front()}; qu.pop();
            for (auto& y : adj[x]) {
                if (base(x) == base(y))
                    continue;
                if (d[y] == -1) {
                    p[y] = x, d[y] = 1;
                    if (m[y] == -1) { // augmenting path
                        augment(-1, y);
                        return true;
                    } else {
                        p[m[y]] = y, d[m[y]] = 0;
                        qu.push(m[y]);
                    }
                } else if (d[base(y)] == 0) { // blossom
                    int b{lca(base(x), base(y))};
                    handle_one_side(x, y, b);
                    handle_one_side(y, x, b);
                }
            }
        }
        return false;
    }
public:
    vector<vector<int>> adj;
    vector<int> m;
    explicit graph(int _n) : n{_n}, p(n, -1), d(n), a(n), c1(n), c2(n), dsu_p(n), dsu_sz(n), dsu_b(n), adj(n), m(n, -1) {}
    int Edmonds() {
        int c{0};
        for (int v{0}; v < n; ++v)
            if (m[v] == -1 && bfs(v)) ++c;
        return c;
    }
};

// usage example:
int main() {
    int n = 5;
    graph g(n);
    auto add = [&](int u, int v) {
        g.adj[u].push_back(v);
        g.adj[v].push_back(u);
    };

    // sample graph
    add(0, 1);
    add(0, 2);
    add(1, 2); // triangle (0,1,2)
    add(2, 3);
    add(3, 4);

    int match_size = g.Edmonds();
    cout << "Maximum matching size: " << match_size << "\n";
    for (int i = 0; i < n; ++i)
        if (g.m[i] != -1 && i < g.m[i])
            cout << i << " - " << g.m[i] << "\n";
}
```

6.12 topological sort

```
// topological sort 1
optional<vector<int>> top_sort(vector<vector<int>>& adj) {
    vector<int> res{};
    int n{static_cast<int>(adj.size())};
    vector<int> cnt(n, 0); // predecessor count
    for (int u = 0; u < n; ++u)
        for (auto& v : adj[u]) ++cnt[v];

    queue<int> qu{};
    for (int u = 0; u < n; ++u) if (!cnt[u]) qu.push(u);
    while (!qu.empty()) {
        auto u = qu.front();
        qu.pop();
        res.push_back(u);

        for (auto& v : adj[u])
            if (!--cnt[v]) qu.push(v);
    }

    if (res.size() != adj.size()) return nullopt;
    return res;
}
```

6.13 Flyod's algorithm

```
// Floyd's Cycle-Finding Algorithm
// Given a function succ(x) that returns the successor of x,
a = succ(x);
b = succ(succ(x));
while (a != b) {
    a = succ(a);
    b = succ(succ(b));
}

// Find the first element in the cycle
a = x;
while (a != b) {
    a = succ(a);
    b = succ(b);
}
first = a;

// Find the length of the cycle
length = 1;
b = succ(a);
while (a != b) {
    b = succ(b);
    length = length + 1;
}

// Finding the starting point of the cycle from any node
int n;
vector<int> adj, res;
vector<vector<int>> radj;

void fill_radj(int u) {
    for (int v : radj[u]) {
        if (res[v] == -1) {
            res[v] = res[u];
            fill_radj(v);
        }
    }
}

void floyd(int x) {
    int y = x;
    do { // find a cycle using Floyd's algorithm
        x = adj[x];
        y = adj[adj[y]];
    } while (y != x);
    do { // set res[x] = x for all x along cycle
        res[x] = x;
        x = adj[x];
    } while (y != x);
    do { // set res'es for all x not along cycle
        fill_radj(x);
        x = adj[x];
    } while (y != x);
}

void solve() {
    cin >> n;
    adj.resize(n);
    radj.assign(n, {});
    res.assign(n, -1);

    for (auto& e : adj) {
        cin >> e;
        e--;
    }

    for (int i = 0; i < n; ++i) {
        radj[adj[i]].push_back(i);
    }

    for (int i = 0; i < n; ++i) {
        if (res[i] == -1) {
            floyd(i);
        }
    }

    for (int i = 0; i < n; ++i) {
        cout << res[i] + 1 << " ";
    }
    cout << "\n";
}
```

6.14 stable matching

```cpp
// Stable Marriage Problem (stable matching)
// Gale – Shapley algorithm
class bipartiteGraph {
public:
    int n;
    vector<vector<int>> px, py;
    vector<int> mx, my;

    bipartiteGraph(int _n) : n(_n), px(n, vector<int>(n)), py(n, vector<int>(n)), mx(n, -1), my(n, -1) {}
    void Gale_Shapley() {
        queue<int> qu{};
        vector<priority_queue<pair<int, int>>> pq(n);
        for (int x{0}; x < n; ++x) {
            qu.push(x);
            for (int y{0}; y < n; ++y) pq[x].emplace(px[x][y], y);
        }

        while (!qu.empty()) {
            auto x{qu.front()}; qu.pop();

            int y;
            do {
                y = pq[x].top().second; pq[x].pop();
            } while (my[y] != -1 && py[y][x] <= py[y][my[y]]);

            if (my[y] != -1) mx[my[y]] = -1, qu.push(my[y]); // y prefers x to my[y]
            mx[x] = y, my[y] = x;
        }
    }
};

int main() {
    int n = 3;
    bipartiteGraph g(n);

    // Men preferences (from most to least preferred)
    vector<vector<int>> men = {
        {0, 1, 2},
        {1, 2, 0},
        {1, 0, 2}
    };

    // Women preferences (from most to least preferred)
    vector<vector<int>> women = {
        {1, 0, 2},
        {2, 1, 0},
        {0, 1, 2}
    };

    // Convert orders to scores (higher is better)
    for (int x = 0; x < n; ++x)
        for (int pos = 0; pos < n; ++pos)
            g.px[x][men[x][pos]] = n - pos;

    for (int y = 0; y < n; ++y)
        for (int pos = 0; pos < n; ++pos)
            g.py[y][women[y][pos]] = n - pos;

    g.Gale_Shapley();

    // Output matches: man x is matched to woman g.mx[x]
    for (int x = 0; x < n; ++x)
        cout << x << " " << g.mx[x] << '\n';

    return 0;
}
```

6.15 Bellman-Ford

```cpp
// Bellman-Ford algorithm
template<typename T>
optional<vector<optional<T>>> Bellman_Ford(const vector<vector<pair<int, T>>>& adj, int s) {
    const auto& n{adj.size()};
    vector<optional<T>> d(n, nullopt);
    d[s] = 0;

    queue<int> qu{}, qu2{};
    vector<bool> in(n, false), in2(n, false);
    qu.push(s), in[s] = true;
    for (int i{0}; i < n; ++i) { // at most n-1 edges
        while (!qu.empty()) {
            int u{qu.front()};
            qu.pop(), in[u] = false;
            for (auto& [v, w] : adj[u])
                if (!d[v] || d[v] > d[u].value() + w) { // relax
                    d[v] = d[u].value() + w;
                    if (!in2[v]) qu2.push(v), in2[v] = true;
                }
        }
        qu.swap(qu2), in.swap(in2);
    }

    if (qu.empty()) return d;
    return nullopt; // if negative cycle
}
```

7 Number Theory

7.1 Linear Sieve

```cpp
// Calculate the smallest divisor of integers in [2, maxn) in O(maxn)
vector<int> min_div{[] {
    constexpr int maxn = 400000 + 10;

    vector<int> v(maxn), p;

    for (int i = 2; i < maxn; ++i) {
        if (!v[i]) {
            v[i] = i;
            p.push_back(i);
        }
        for (int j = 0; p[j] * i < maxn; ++j) {
            v[p[j] * i] = p[j];
            if (p[j] == v[i]) break;
        }
    }

    return v;
}()};
```

7.2 C(n,m)

```cpp
ll Cnm(ll n, ll m) {
    if (m > n / 2) m = n - m;
    ll r{1};
    for (ll i{1}, j{n}; i <= m; ++i, --j) r *= j, r /= i;
    return r;
}
```

7.3 Euler Totient 1 to n

```cpp
void phi_1_to_n(int n) {
    vector<int> phi(n + 1);
    for (int i = 0; i <= n; i++)
        phi[i] = i;

    for (int i = 2; i <= n; i++) {
        if (phi[i] == i) {
            for (int j = i; j <= n; j += i)
                phi[j] -= phi[j] / i;
        }
    }
}
```

7.4 derangement (Principle of Inclusion-Exclusion)

```cpp
// 1. Principle of Inclusion-Exclusion
// n! = n! * Σ (from k=0 to n) [((-1)^k) / (k!)]

mint c = 1;
for (int i = 1; i <= n; i++) {
    c = (c * i) + (i % 2 == 1 ? -1 : 1);
    cout << c.val() << ' ';
}
```

7.5 matrix template (with fast power)

```cpp
template<class T> struct Matrix {
    T **mat; int a, b;

    Matrix() : a(0), b(0) {}
    Matrix(int a_, int b_) : a(a_), b(b_) {
        int i, j;
        mat = new T*[a];
        for (i = 0; i < a; ++i) {
            mat[i] = new T[b];
            for (j = 0; j < b; ++j ){
                mat[i][j] = 0;
            }
        }
    }
    Matrix(const vector<vector<T>>& vt) {
        int i, j;

        *this = Matrix((int)vt.size(), (int)vt[0].size());
        for (i = 0; i < a; ++i) {
            for (j = 0; j < b; ++j) {
                mat[i][j] = vt[i][j];
            }
        }
    }
    Matrix operator*(const Matrix& m) {
        int i, j, k;

        assert(b == m.a);
        Matrix r(a, m.b);
        for (i = 0; i < a; ++i) {
            for (j = 0; j < m.b; ++j) {
                for (k = 0; k < b; ++k) {
                    r.mat[i][j] += mul(mat[i][k], m.mat[k][j]);
                    r.mat[i][j] %= MOD;
                }
            }
        }
        return r;
    }
    Matrix& operator*=(const Matrix& m) {
        return *this = (*this) * m;
    }
    friend Matrix power(Matrix m, long long p) {
        int i;

        assert(m.a == m.b);
        Matrix r(m.a, m.b);
        for (i = 0; i < m.a; ++i) {
            r.mat[i][i] = 1;
        }
        for (; p > 0; p >>= 1, m *= m) {
            if (p & 1) {
                r *= m;
            }
        }
        return r;
    }
};
```

```
int main() {
    Matrix<int> mat(adj);
    mat = power(mat, k); // mat^k
    cout << mat.mat[i][j];
}
```

7.6 Sieve of Eratosthenes (with big num)

```
const int MX = 100000;
bool np[MX + 1];
vector<int> prime_numbers;

int init = []() {
    np[0] = np[1] = true;
    for (int i = 2; i <= MX; i++) {
        if (!np[i]) {
            prime_numbers.push_back(i);
            for (int j = i; j <= MX / i; j
                ++) { // 避免溢出的写法
                np[i * j] = true;
            }
        }
    }
    return 0;
}();

bool is_prime(long long n) {
    if (n <= MX) {
        return !np[n];
    }
    for (long long p : prime_numbers) {
        if (p * p > n) {
            break;
        }
        if (n % p == 0) {
            return false;
        }
    }
    return true;
}
```

7.7 mod inv

```
// Modular inverse when mod is prime
long long mod_inverse(long long a, long long
    mod) {
    return power_mod(a, mod - 2, mod);  //
        Fermat's Little Theorem
}
```

7.8 derangement (DP)

```
// 2. DP
// !n = (n - 1) * (!(n - 1) + !(n - 2)),
    with  !0 = 1,  !1 = 0

mint a = 1, b = 0;
cout << 0 << ' ';

for (int i = 2; i <= n; i++) {
    mint c = (i - 1) * (a + b);
    cout << c.val() << ' ';
    a = b;
    b = c;
}
```

7.9 Euler Totient

```
int phi(int n) {
    int result = n;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            while (n % i == 0)
                n /= i;
            result -= result / i;
        }
    }
    if (n > 1)
        result -= result / n;
    return result;
}
```

7.10 fast power

```
/* Iterative Function to calculate (a^b) %
    mod in O(Log b) */
long long power_mod(long long a, long long b
    , long long mod) {
    long long res{1};
    while (b > 0) {
        if (b & 1) res = res * a % mod;
        b >>= 1;
        a = (a * a) % mod;
    }
    return res;
}
```

7.11 first and second mex

```
// Calculate first and second MEX
pair<int, int> calculate_mexes(vector<int>&
    nums) {
    int n = nums.size();
    vector<bool> seen(n + 2, false);

    for (int num : nums) {
        if (num >= 0 && num < seen.size()) {
            seen[num] = true;
        }
    }

    int first_mex = -1;
    int second_mex = -1;

    for (int i = 0; i < seen.size(); ++i) {
        if (!seen[i]) {
            if (first_mex == -1) {
                first_mex = i;
            } else {
                second_mex = i;
                break;
            }
        }
    }

    return {first_mex, second_mex};
}
```

7.12 Catalan Number

```
// Catalan Number
// C(n) = 1/(n+1) * (2n choose n) = (2n)! /
    ((n+1)! * n!)

ll catalan(int n) {
    return fact[2 * n] * invf[n + 1] % mod *
        invf[n] % mod;
}
```

7.13 Chinese Remainder Theorem

```
// coprime (p^k)
pair<ll, ll> CRT(const vector<pair<ll, ll>>&
    congruences) {
    ll M = 1, sol = 0;
    for (auto& [m, a] : congruences) M *= m;
    for (auto& [m, a] : congruences) {
        ll x = M / m, y = MI(x, m);
        sol = MA(sol, MM(MM(a, x, M), y, M),
            M);
    }
    return {M, sol};
}

// example usage
int main() {
    // x ≡ 2 (mod 3)
    // x ≡ 3 (mod 5)
    // x ≡ 2 (mod 7)
    vector<pair<ll, ll>> congruences = {{3,
        2}, {5, 3}, {7, 2}};
    auto [M, sol] = CRT(congruences);
    cout << "M: " << M << ", x: " << sol <<
        endl; // M: 105, x: 23
    return 0;
}
```

7.14 mod inv (not prime)

```
/* exists when a and mod are coprime */
/* but mod is not prime */
long long MI(long long a, long long mod) {
    return power_mod(a, euler_phi(mod) - 1,
        mod);
}
```

7.15 mod inv (not coprime)

```
/* a and mod are not coprime */
long long MI(long long a, long long mod) {
    long long d, x, y;
    extEcu(a, mod, d, x, y);
    return d == 1 ? (x + mod) % mod : -1;
}
```

7.16 inclusion-exclusion principle

```
/*
includsion-exclusion principle: U A = Σ |A_i
    | - Σ |A_i ∩ A_j| + Σ |A_i ∩ A_j ∩ A_k|
    - ...

if f(n) = Σ g(d) for all d|n
then g(n) = Σ μ(d) * f(n/d) for all d|n

where μ(d) is the mobius function defined as
    :

|U A| = Σ μ(d) * |A_d|
where A_d = {x ∈ U : d|x}
μ(d) = 1 if d is a product of even number of
    distinct primes
μ(d) = -1 if d is a product of odd number of
    distinct primes
μ(d) = 0 if d has a squared prime factor
*/

/*
problem: In number theory we call an integer
    square-free if it is not divisible by a
    perfect square, except 1. You have to
    count them!
square-free number: An integer n is square-
    free if no p^2 divides n.
*/
const int VALMAX = 1e7;

int mobius[VALMAX];

int main() {
  int test_num;
  cin >> test_num;

  mobius[1] = -1;
  for (int i = 1; i < VALMAX; i++) {
    if (mobius[i]) {
```

```
      mobius[i] = -mobius[i];
      for (int j = 2 * i; j < VALMAX; j += i
           ) { mobius[j] += mobius[i]; }
    }
  }

  for (int t = 0; t < test_num; t++) {
    long long n;
    long long ans = 0;
    cin >> n;
    for (int i = 1; 1LL * i * i <= n; i++) {
      ans += mobius[i] * n / ((long long)i *
           i);
    }
    cout << ans << '\n';
  }
}

/*
problem: Given N cows (2 <= N <= 50,000),
     each listing five favorite ice cream
     flavors,
 where two cows are compatible if they share
      at least one flavor,
determine the number of cow pairs that are
     not compatible.

ans = n * (n - 1) / 2 - Σ |A_i| + Σ |A_i ∩
     A_j| - Σ |A_i ∩ A_j ∩ A_k| + ...
where A_i is the set of cows that like
     flavor i
*/

void solve() {
    int n;
    cin >> n;
    vector<array<int, 5>> vc(n);

    map<vector<int>, int> subsets;
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < 5; ++j) {
            cin >> vc[i][j];
        }

        sort(vc[i].begin(), vc[i].end());
        for (int mask = 1; mask < (1<<5); ++
             mask) {
            vector<int> subset;
            for (int b = 0; b < 5; ++b) {
                if (mask & (1<<b)) {
                    subset.push_back(vc[i][b
                         ]);
                }
            }
            subsets[subset]++;
        }
    }

    int ans = n * (n - 1) / 2;
    for (const auto& [subset, freq] :
         subsets) {
        ans -= (subset.size() % 2 == 1 ? 1 :
             -1) * freq * (freq - 1) / 2;
    }
    cout << ans << "\n";
}
```

7.17 C(n,k) mod inverse

```
fac[0] = 1;
for (int i = 1; i <= n; ++i) {
    fac[i] = fac[i - 1] * i % MOD;
}

inv_fac[n] = power_mod(fac[n], MOD - 2, MOD)
    ;
for (int i = n - 1; i >= 0; --i) {
    inv_fac[i] = inv_fac[i + 1] * (i + 1) %
         MOD;
}

// C(n, k) = fac[n] * inv_fac[k] * inv_fac[n
     - k];
```

7.18 Linear Diophantine 丟番圖

```
// ax + by = c
// x0 = x * (c / gcd(a, b))
// y0 = y * (c / gcd(a, b))
// x = x0 + k * (b / gcd(a, b))
// y = y0 - k * (a / gcd(a, b))
// k is any integer

tll extendedEuclid(ll a, ll b) {
    if (b == 0) return {a, 1, 0};
    ll d, x1, y1;
    tie(d, x1, y1) = extendedEuclid(b, a % b
        );
    return {d, y1, x1 - (a / b) * y1};
}

pll linearDiophantine(ll a, ll b, ll c) {
    ll d, x, y;
    tie(d, x, y) = extendedEuclid(a, b);
    if (c % d != 0) return {-1, -1}; // no
        solution
    x *= c / d;
    y *= c / d;
    return {x, y};
}

// Example usage
int main() {
    ll a = 15, b = 10, c = 35;
    pll res = linearDiophantine(a, b, c);
    if (res.first == -1 && res.second == -1)
        {
        cout << "No solution exists" << endl
            ;
    } else {
        cout << "One solution is x = " <<
            res.first << ", y = "
            << res.second << endl;
    }
    return 0;
}
ax + by = c
```

7.19 擴展歐基里德

```
/* solve x, y for ax + by = gcd(a, b) = g */
template<typename T>
void extEcu(T a, T b, T &g, T &x, T &y) {
    if (b) extEcu(b, a % b, g, y, x), y -= x
        * (a / b);
    else g = a, x = 1, y = 0;
}
```

7.20 Sieve of Eratosthenes

```
void sieve(vector<int>& primes) {
    vector<int> is_prime(INF + 1, 0);
    is_prime[0] = 1;
    is_prime[1] = 1;

    int sq = sqrt(INF);

    for (int i = 2; i <= sq; ++i) {
        if (!is_prime[i]) {
            primes.push_back(i);
            for (int j = i * i; j <= INF; j
                 += i) {
                is_prime[j] = 1;
            }
        }
    }
}
```

7.21 builtin

```
// __builtin_ctz
// 這個函數作用是返回輸入數二進制表示從最低
     位開始（右起）的連續的0的個數；如果傳入0
     則行為未定義。
int __builtin_ctz (unsigned int x)

// __builtin_clz
// 這個函數作用是返回輸入數二進制表示從最高
     位開始（左起）的連續的0的個數；如果傳入0
     則行為未定義。
int __builtin_clz (unsigned int x)

// __builtin_ffs
//  這個函數作用是返回輸入數二進制表示的最低
     非0位的下標，下標從1開始計數；如果傳入0
     則返回0
int __builtin_ffs (unsigned int x)

// __builtin_popcount
// 這個函數作用是返回輸入數二進制表示中1的個
     數。
int __builtin_popcount (unsigned int x)

// __builtin_parity
// 這個函數作用是返回輸入數二進制表示中1的個
     數的奇偶性，1表示奇數個1，0表示偶數個1。
int __builtin_parity (unsigned int x)
```

7.22 C(n,k) DP

```
long long binomial(long long n, long long k,
     long long p) {
    // dp[i][j] = iCj
    vector<vector<long long>> dp(n + 1,
        vector<long long>(k + 1, 0));

    for (int i = 0; i <= n; ++i) {
        dp[i][0] = 1;
        if (i <= k) dp[i][i] = 1;
    }

    for (int i = 0; i <= n; ++i) {
        for (int j = 1; j <= min(i, k); ++j)
            {
            if (i != j) {
                dp[i][j] = (dp[i - 1][j - 1]
                    + dp[i - 1][j]) % p;
            }
        }
    }

    return dp[n][k];
}
```

8 Range Queries

8.1 subarray maximum sum queries

```
// CSES: Subarray Sum Queries

const int maxn = 2e5;
int n = 0;
int t[maxn << 1];

void build() {
    for (int i = 0; i < n; ++i) {
        t[i + n] = 0;
    }
    for (int i = n - 1; i >= 1; --i) {
        t[i] = t[i << 1] + t[i << 1 | 1];
    }
}

void update(int idx, int val) {
    idx += n;
    if (t[idx] == val) return;
    t[idx] = val;
    for (idx >>= 1; idx >= 1; idx >>= 1) {
        t[idx] = t[idx << 1] + t[idx << 1 |
            1];
    }
}
```

```

int query(int l, int r) {
    int ans = 0;
    for (l += n, r += n; l < r; l >>= 1, r
         >>= 1) {
        if (l & 1) ans += t[l++];
        if (r & 1) ans += t[--r];
    }
    return ans;
}

void solve() {
    int q;
    cin >> q;
    n = 0;
    vector<pii> op(q);
    map<int, int> mp;
    vector<int> to_val;

    memset(t, 0, sizeof(t));

    for (int i = 0; i < q; ++i) {
        char c;
        cin >> c >> op[i].second;
        if (c == 'I') {
            op[i].first = 0;
            mp[op[i].second] = 1;
        }
        else if (c == 'D') {
            op[i].first = 1;
            mp[op[i].second] = 1;
        }
        else if (c == 'K') {
            op[i].first = 2;
        }
        else {
            op[i].first = 3;
        }
    }

    for (auto& x : mp) {
        x.second = n;
        to_val.push_back(x.first);
        n++;
    }

    build();

    for (pii o : op) {
        if (o.first == 0) {
            int idx = mp[o.second];
            update(idx, 1);
        }
        if (o.first == 1) {
            int idx = mp[o.second];
            update(idx, 0);
        }
        if (o.first == 2) {
            int k = o.second;
            int ans = - 1;
            int l = 0, r = n - 1;
            while (l <= r) {
                int m = l + (r - l) / 2;
                int rk = query(0, m + 1);
                if (rk >= k) {
                    ans = m;
                    r = m - 1;
                }
                else {
                    l = m + 1;
                }
            }
            if (ans == -1) cout << "invalid\
n";
            else cout << to_val[ans] << "\n"
                ;
        }
        if (o.first == 3) {
            auto it = mp.lower_bound(o.
                second);
            if (it == mp.begin() && (it ==
                mp.end() || o.second <= it->
                first)) {
                cout << "0\n";
            }
            else if (it == mp.end()) {
                cout << query(0, n) << "\n";
            }
            else {
                cout << query(0, it->second)
                     << "\n";
            }

        }
    }
}
```

8.2 find first equal or greater

```
// CSE: Hotel Queries

int n, m;
int sz = 1;
const int maxn = 2e6;
int t[maxn << 1];

void build(const vector<int>& a) {
    for (int i = 0; i < n; ++i) {
        t[i + sz] = a[i];
    }
    for (int i = sz - 1; i >= 1; --i) {
        t[i] = max(t[i << 1], t[i << 1 | 1])
            ;
    }
}

void update(int i, int val) {
    i += sz;
    t[i] = val;
    for (i >>= 1; i >= 1; i >>= 1) {
        t[i] = max(t[i << 1], t[i << 1 | 1])
            ;
    }
}

int query(int r) {
    if (t[1] < r) return 0;

    int i = 1;
    while (i < sz) {
        if (t[i << 1] >= r) i = i << 1;
        else i = i << 1 | 1;
    }
    return i - sz + 1;
}

void solve() {
    cin >> n >> m;
    vector<int> a(n);

    for (int i = 0; i < n; ++i) {
        cin >> a[i];
    }

    while (sz < n) sz <<= 1;
    memset(t, 0, sizeof(t));
    build(a);

    int r;
    while (m--) {
        cin >> r;
        int idx = query(r);
        cout << idx << " ";
        if (idx) {
            a[idx - 1] -= r;
            update(idx - 1, a[idx - 1]);
        }
    }
    cout << "\n";
}
```

8.3 find k-th and count less than

```
/*
SPOJ: ORDERSET - Order statistic set

In this problem, you have to maintain a
     dynamic set of numbers which support the
      two fundamental operations

INSERT(S,x): if x is not in S, insert x into
      S
DELETE(S,x): if x is in S, delete x from S
and the two type of queries

K-TH(S) : return the k-th smallest element
     of S
COUNT(S,x): return the number of elements of
      S smaller than x
*/

const int maxn = 2e5;
int n = 0;
int t[maxn << 1];

void build() {
    for (int i = 0; i < n; ++i) {
        t[i + n] = 0;
    }
    for (int i = n - 1; i >= 1; --i) {
        t[i] = t[i << 1] + t[i << 1 | 1];
    }
}

void update(int idx, int val) {
    idx += n;
    if (t[idx] == val) return;
    t[idx] = val;
    for (idx >>= 1; idx >= 1; idx >>= 1) {
        t[idx] = t[idx << 1] + t[idx << 1 |
            1];
    }
}

int query(int l, int r) {
    int ans = 0;
    for (l += n, r += n; l < r; l >>= 1, r
         >>= 1) {
        if (l & 1) ans += t[l++];
        if (r & 1) ans += t[--r];
    }
    return ans;
}

void solve() {
    int q;
    cin >> q;
    n = 0;
    vector<pii> op(q);
    map<int, int> mp;
    vector<int> to_val;

    memset(t, 0, sizeof(t));

    for (int i = 0; i < q; ++i) {
        char c;
        cin >> c >> op[i].second;
        if (c == 'I') {
            op[i].first = 0;
            mp[op[i].second] = 1;
        }
        else if (c == 'D') {
            op[i].first = 1;
            mp[op[i].second] = 1;
        }
        else if (c == 'K') {
            op[i].first = 2;
        }
        else {
            op[i].first = 3;
        }
    }

    for (auto& x : mp) {
        x.second = n;
        to_val.push_back(x.first);
        n++;
    }

    build();

    for (pii o : op) {
        if (o.first == 0) {
            int idx = mp[o.second];
            update(idx, 1);
        }
        if (o.first == 1) {
            int idx = mp[o.second];
```

```
            update(idx, 0);
        }
        if (o.first == 2) {
            int k = o.second;
            int ans = - 1;
            int l = 0, r = n - 1;
            while (l <= r) {
                int m = l + (r - l) / 2;
                int rk = query(0, m + 1);
                //printf("(m, rk): %lld, %
                    lld\n", m, rk);
                if (rk >= k) {
                    ans = m;
                    r = m - 1;
                }
                else {
                    l = m + 1;
                }
            }
            if (ans == -1) cout << "invalid\
                n";
            else cout << to_val[ans] << "\n"
                ;
        }
        if (o.first == 3) {
            auto it = mp.lower_bound(o.
                second);
            if (it == mp.begin() && (it ==
                mp.end() || o.second <= it->
                first)) {
                cout << "0\n";
            }
            else if (it == mp.end()) {
                cout << query(0, n) << "\n";
            }
            else {
                cout << query(0, it->second)
                    << "\n";
            }

        }
    }
}
```

9 String

9.1 Z

```
// 計算并返回 z 数组，其中 z[i] = |LCP(s[i
    :], s)|
vector<int> calc_z(const string& s) {
    int n = s.size();
    vector<int> z(n);
    int box_l = 0, box_r = 0;
    for (int i = 1; i < n; i++) {
        if (i <= box_r) {
            z[i] = min(z[i - box_l], box_r -
                i + 1);
        }
        while (i + z[i] < n && s[z[i]] == s[
            i + z[i]]) {
            box_l = i;
            box_r = i + z[i];
            z[i]++;
        }
    }
    z[0] = n;
    return z;
}
```

9.2 manacher

```
class Solution {
public:
    int countSubstrings(string s) {
        int l1 = s.size(), l2 = l1 * 2 + 1;
        string ch = "#";
        for(char c: s) {
            ch = ch + c + "#";
        }

        int c = 0, r = 0, cnt = 0;
        vector<int> p(l2);
        for(int i = 0; i < l2; i++) {
            p[i] = (i < r)? min(p[2 * c - i
                ], r - i): 1;
            while(i + p[i] < l2 && i - p[i]
                >= 0 && ch[i + p[i]] == ch[i
                - p[i]]) p[i]++;
            if(i + p[i] > r) {
                r = i + p[i];
                c = i;
            }
            int l = p[i] - 1;
            if(l % 2 == 0) cnt += l / 2;
            else cnt += l / 2 + 1;
        }
        return cnt;
    }
};
```

9.3 AC 自動機

```
template<char L='a',char R='z'>
class ac_automaton{
  struct joe{
    int next[R-L+1],fail,efl,ed,cnt_dp,vis;
    joe():ed(0),cnt_dp(0),vis(0){
      for(int i=0;i<=R-L;++i)next[i]=0;
    }
  };
public:
  std::vector<joe> S;
  std::vector<int> q;
  int qs,qe,vt;
  ac_automaton():S(1),qs(0),qe(0),vt(0){}
  void clear(){
    q.clear();
    S.resize(1);
    for(int i=0;i<=R-L;++i)S[0].next[i]=0;
    S[0].cnt_dp=S[0].vis=qs=qe=vt=0;
  }
  void insert(const char *s){
    int o=0;
    for(int i=0,id;s[i];++i){
      id=s[i]-L;
      if(!S[o].next[id]){
        S.push_back(joe());
        S[o].next[id]=S.size()-1;
      }
      o=S[o].next[id];
    }
    ++S[o].ed;
  }
  void build_fail(){
    S[0].fail=S[0].efl=-1;
    q.clear();
    q.push_back(0);
    ++qe;
    while(qs!=qe){
      int pa=q[qs++],id,t;
      for(int i=0;i<=R-L;++i){
        t=S[pa].next[i];
        if(!t)continue;
        id=S[pa].fail;
        while(~id&&!S[id].next[i])id=S[id].
            fail;
        S[t].fail=~id?S[id].next[i]:0;
        S[t].efl=S[S[t].fail].ed?S[t].fail:S
            [S[t].fail].efl;
        q.push_back(t);
        ++qe;
      }
    }
  }
  /*DP出每個前綴在字串s出現的次數並傳回所有
      字串被s匹配成功的次數O(N+M)*/
  int match_0(const char *s){
    int ans=0,id,p=0,i;
    for(i=0;s[i];++i){
      id=s[i]-L;
      while(!S[p].next[id]&&p)p=S[p].fail;
      if(!S[p].next[id])continue;
      p=S[p].next[id];
      ++S[p].cnt_dp;/*匹配成功則它所有後綴都
          可以被匹配(DP計算)*/
    }
    for(i=qe-1;i>=0;--i){
      ans+=S[q[i]].cnt_dp*S[q[i]].ed;
      if(~S[q[i]].fail)S[S[q[i]].fail].
          cnt_dp+=S[q[i]].cnt_dp;
    }
    return ans;
  }
  /*多串匹配走efl邊並傳回所有字串被s匹配成功
      的次數O(N*M^1.5)*/
  int match_1(const char *s)const{
    int ans=0,id,p=0,t;
    for(int i=0;s[i];++i){
      id=s[i]-L;
      while(!S[p].next[id]&&p)p=S[p].fail;
      if(!S[p].next[id])continue;
      p=S[p].next[id];
      if(S[p].ed)ans+=S[p].ed;
      for(t=S[p].efl;~t;t=S[t].efl){
        ans+=S[t].ed;/*因為都走efl邊所以保證
            匹配成功*/
      }
    }
    return ans;
  }
  /*枚舉(s的子字串nA)的所有相異字串各恰一次
      並傳回次數O(N*M^(1/3))*/
  int match_2(const char *s){
    int ans=0,id,p=0,t;
    ++vt;
    /*把戳記vt+=1，只要vt沒溢位，所有S[p].
        vis==vt就會變成false
    這種利用vt的方法可以O(1)歸零vis陣列*/
    for(int i=0;s[i];++i){
      id=s[i]-L;
      while(!S[p].next[id]&&p)p=S[p].fail;
      if(!S[p].next[id])continue;
      p=S[p].next[id];
      if(S[p].ed&&S[p].vis!=vt){
        S[p].vis=vt;
        ans+=S[p].ed;
      }
      for(t=S[p].efl;~t&&S[t].vis!=vt;t=S[t
          ].efl){
        S[t].vis=vt;
        ans+=S[t].ed;/*因為都走efl邊所以保證
            匹配成功*/
      }
    }
    return ans;
  }
  /*把AC自動機變成真的自動機*/
  void evolution(){
    for(qs=1;qs!=qe;){
      int p=q[qs++];
      for(int i=0;i<=R-L;++i)
        if(S[p].next[i]==0)S[p].next[i]=S[S[
            p].fail].next[i];
    }
  }
};
```

9.4 KMP

```
// 在文本串 text 中查找模式串 pattern，返回
    所有成功匹配的位置（pattern[0] 在 text
    中的下标）
vector<int> kmp(const string& text, const
    string& pattern) {
    int m = pattern.size();
    vector<int> pi(m);
    int cnt = 0;
    for (int i = 1; i < m; i++) {
        char b = pattern[i];
        while (cnt && pattern[cnt] != b) {
            cnt = pi[cnt - 1];
        }
        if (pattern[cnt] == b) {
            cnt++;
        }
```

```
        pi[i] = cnt;
    }

    vector<int> pos;
    cnt = 0;
    for (int i = 0; i < text.size(); i++) {
        char b = text[i];
        while (cnt && pattern[cnt] != b) {
            cnt = pi[cnt - 1];
        }
        if (pattern[cnt] == b) {
            cnt++;
        }
        if (cnt == m) {
            pos.push_back(i - m + 1);
            cnt = pi[cnt - 1];
        }
    }
    return pos;
}
```

9.5 suffix array lcp

```
#define radix_sort(x,y){\
  for(i=0;i<A;++i)c[i]=0;\
  for(i=0;i<n;++i)c[x[y[i]]]++;\
  for(i=1;i<A;++i)c[i]+=c[i-1];\
  for(i=n-1;~i;--i)sa[--c[x[y[i]]]]=y[i];\
}
#define AC(r,a,b)\
  r[a]!=r[b]||a+k>=n||r[a+k]!=r[b+k]
void suffix_array(const char *s,int n,int *
    sa,int *rank,int *tmp,int *c){
  int A='z'+1,i,k,id=0;
  for(i=0;i<n;++i)rank[tmp[i]=i]=s[i];
  radix_sort(rank,tmp);
  for(k=1;id<n-1;k<<=1){
    for(id=0,i=n-k;i<n;++i)tmp[id++]=i;
    for(i=0;i<n;++i)
      if(sa[i]>=k)tmp[id++]=sa[i]-k;
    radix_sort(rank,tmp);
    swap(rank,tmp);
    for(rank[sa[0]]=id=0,i=1;i<n;++i)
      rank[sa[i]]=id+=AC(tmp,sa[i-1],sa[i]);
    A=id+1;
  }
}
//h:高度數組 sa:後綴數組 rank:排名
void suffix_array_lcp(const char *s,int len,
    int *h,int *sa,int *rank){
  for(int i=0;i<len;++i)rank[sa[i]]=i;
  for(int i=0,k=0;i<len;++i){
    if(rank[i]==0)continue;
    if(k)--k;
    while(s[i+k]==s[sa[rank[i]-1]+k])++k;
    h[rank[i]]=k;
  }
  h[0]=0;// h[k]=lcp(sa[k],sa[k-1]);
}
```

9.6 hash

```
#define MAXN 1000000
#define mod 1073676287
/*mod 必須要是質數*/
typedef long long T;
char s[MAXN+5];
T h[MAXN+5];/*hash陣列*/
T h_base[MAXN+5];/*h_base[n]=(prime^n)%mod*/
void hash_init(int len,T prime){
  h_base[0]=1;
  for(int i=1;i<=len;++i){
    h[i]=(h[i-1]*prime+s[i-1])%mod;
    h_base[i]=(h_base[i-1]*prime)%mod;
  }
}
T get_hash(int l,int r){/*閉區間寫法，設編號
    為0 ~ len-1*/
  return (h[r+1]-(h[l]*h_base[r-l+1])%mod+
      mod)%mod;
}
```

9.7 minimal string rotation

```
int min_string_rotation(const string &s){
  int n=s.size(),i=0,j=1,k=0;
  while(i<n&&j<n&&k<n){
    int t=s[(i+k)%n]-s[(j+k)%n];
    ++k;
    if(t){
      if(t>0)i+=k;
      else j+=k;
      if(i==j)++j;
      k=0;
    }
  }
  return min(i,j);//最小循環表示法起始位置
}
```

9.8 reverseBWT

```
const int MAXN = 305, MAXC = 'Z';
int ranks[MAXN], tots[MAXC], first[MAXC];
void rankBWT(const string &bw){
  memset(ranks,0,sizeof(int)*bw.size());
  memset(tots,0,sizeof(tots));
  for(size_t i=0;i<bw.size();++i)
    ranks[i] = tots[int(bw[i])]++;
}
void firstCol(){
  memset(first,0,sizeof(first));
  int totc = 0;
  for(int c='A';c<='Z';++c){
    if(!tots[c]) continue;
    first[c] = totc;
    totc += tots[c];
  }
}
string reverseBwt(string bw,int begin){
  rankBWT(bw), firstCol();
  int i = begin; //原字串最後一個元素的位置
  string res;
  do{
    char c = bw[i];
    res = c + res;
    i = first[int(c)] + ranks[i];
  }while( i != begin );
  return res;
}
```

10 Tree

10.1 Euler tour+RMQ

```
// Euler Tour Technique
class LCA {
    const vector<vector<int>>& adj;
    int n;
    vector<int> d, first, euler{}, log2{};
    vector<vector<int>> st{};
    void dfs(int u, int w = -1, int dep = 0)
        {
        d[u] = dep;
        first[u] = euler.size();
        euler.push_back(u);
        for (auto& v : adj[u]) {
            if (v == w) continue;
            dfs(v, u, dep + 1);
            euler.push_back(u);
        }
    }
public:
    LCA(const vector<vector<int>>& _adj, int
        root) : adj{_adj}, n{adj.size()}, d
        (n), first(n) {
        dfs(root);

        int tn{euler.size()};
        log2.resize(tn + 1);
        log2[1] = 0;
        for (int i{2}; i <= tn; ++i) log2[i]
            = log2[i / 2] + 1;

        st.assign(tn, vector<int>(log2[tn] +
            1));
        for (int i{tn - 1}; i >= 0; --i) {
            st[i][0] = euler[i];
            for (int j{1}; i + (1 << j) <=
                tn; ++j) {
                auto& x{st[i][j - 1]};
                auto& y{st[i + (1 << (j - 1)
                    )][j - 1]};
                st[i][j] = d[x] <= d[y] ? x
                    : y;
            }
        }
    }
    int operator()(int u, int v) {
        int l{first[u]}, r{first[v]};
        if (l > r) swap(l, r);
        ++r; // make the interval left
            closed right open

        int j{log2[r - l]};
        auto& x{st[l][j]};
        auto& y{st[r - (1 << j)][j]};
        return d[x] <= d[y] ? x : y;
    }
};

int main() {
    int n, q;
    cin >> n >> q;
    vector<vector<int>> adj(n);

    for (int i = 1; i < n; ++i) {
        int u;
        cin >> u;
        u--;
        adj[u].pb(i);
        adj[i].pb(u);
    }

    LCA lca(adj, 0);

    while (q--) {
        int u, v;
        cin >> u >> v;
        u--, v--;
        cout << lca(u, v) + 1 << "\\n";
    }
    return 0;
}
```

10.2 HLD template

```
struct HLD {
    int n, cur = 0;
    vector<int> sz, top, dep, par, tin, tout
        , seq;
    vector<vector<int>> adj;
    HLD(int n = 0) {
        init(n);
    }
    void init(int n_) {
        n = n_;
        sz.assign(n, 1);
        top.assign(n, -1);
        dep.assign(n, 0);
        par.assign(n, -1);
        tin.assign(n, 0);
        tout.assign(n, 0);
        seq.assign(n, 0);
        adj.resize(n, {});
    }
    void add_edge(int u, int v) { adj[u].
        push_back(v), adj[v].push_back(u); }
    void build(int root = 0) {
        top[root] = root, dep[root] = 0, par
            [root] = -1;
        dfs1(root), dfs2(root);
    }
```

```
    void dfs1(int u) {
        if (auto it = find(adj[u].begin(),
             adj[u].end(), par[u]); it != adj
             [u].end()) {
            adj[u].erase(it);
        }
        for (auto &v : adj[u]) {
            par[v] = u;
            dep[v] = dep[u] + 1;
            dfs1(v);
            sz[u] += sz[v];
            if (sz[v] > sz[adj[u][0]]) {
                swap(v, adj[u][0]); }
        }
    }
    void dfs2(int u) {
        tin[u] = cur++;
        seq[tin[u]] = u;
        for (auto v : adj[u]) {
            top[v] = v == adj[u][0] ? top[u]
                 : v;
            dfs2(v);
        }
        tout[u] = cur;
    }
    int lca(int u, int v) {
        while (top[u] != top[v]) {
            if (dep[top[u]] > dep[top[v]]) {
                u = par[top[u]];
            } else {
                v = par[top[v]];
            }
        }
        return dep[u] < dep[v] ? u : v;
    }
    int dist(int u, int v) { return dep[u] +
          dep[v] - 2 * dep[lca(u, v)]; }
    int jump(int u, int k) {
        if (dep[u] < k) { return -1; }
        int d = dep[u] - k;
        while (dep[top[u]] > d) { u = par[
            top[u]]; }
        return seq[tin[u] - dep[u] + d];
    }
    // u is v's ancestor
    bool is_ancestor(int u, int v) { return
         tin[u] <= tin[v] && tin[v] < tout[u
         ]; }
    // root's parent is itself
    int rooted_parent(int r, int u) {
        if (r == u) { return u; }
        if (is_ancestor(r, u)) { return par[
            u]; }
        auto it = upper_bound(adj[u].begin()
            , adj[u].end(), r, [&](int x,
            int y) {
            return tin[x] < tin[y];
        }) - 1;
        return *it;
    }
    // rooted at u, v's subtree size
    int rooted_size(int r, int u) {
        if (r == u) { return n; }
        if (is_ancestor(u, r)) { return sz[u
            ]; }
        return n - sz[rooted_parent(r, u)];
    }
    int rooted_lca(int r, int a, int b) {
        return lca(a, b) ^ lca(a, r) ^ lca(b
            , r); }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n = 7;
    HLD hld(n);

    // build this tree:
    //     0
    //    / \
    //   1   2
    //  / \   \
    // 3   4   5
    //         |
    //         6
    hld.add_edge(0, 1);
    hld.add_edge(0, 2);
    hld.add_edge(1, 3);
    hld.add_edge(1, 4);
    hld.add_edge(2, 5);
    hld.add_edge(5, 6);

    hld.build(0); // root at node 0

    cout << "LCA(3,4) = " << hld.lca(3,4) <<
         "\n"; // expected 1
    cout << "LCA(3,6) = " << hld.lca(3,6) <<
         "\n"; // expected 0
    cout << "Distance(3,4) = " << hld.dist
        (3,4) << "\n"; // expected 2
    cout << "Distance(3,6) = " << hld.dist
        (3,6) << "\n"; // expected 4
    cout << "Is 0 ancestor of 6? " << (hld.
        is_ancestor(0,6) ? "yes" : "no") <<
        "\n"; // yes

    return 0;
}
```

10.3 all longest path dfs

```
// all longest path (generalization of the
    tree diameter problem)
vector<tuple<int, int, int>> dp{};
// [mx1, x, mx2] the path of mx1 goes
    through x

int dfs1(int u, int p = -1) {
    int mx = 0;
    for (int v : adj[u]) {
        if (v != p) {
            int len = 1 + dfs1(v, u);
            mx = max(mx, len);

            auto& [mx1, x, mx2] = dp[u];
            if (len >= mx1) {
                mx2 = mx1;
                mx1 = len;
                x = v;
            }
            else if (len > mx2) {
                mx2 = len;
            }
        }
    }
    return mx;
}

void dfs2(int u, int p = -1) {
    if (p != -1) {
        int tmx;
        {   // calculate the longest path
            through parent
            auto& [mx1, x, mx2] = dp[p];
            if (x != u) tmx = mx1 + 1;
            else tmx = mx2 + 1;
        }
        {   // update the path
            auto& [mx1, x, mx2] = dp[u];
            if (tmx >= mx1) {
                mx2 = mx1;
                mx1 = tmx;
                x = p;
            }
            else if (tmx > mx2) {
                mx2 = tmx;
            }
        }
    }
    for (int v : adj[u]) {
        if (v != p) {
            dfs2(v, u);
        }
    }
}

void all_longest_path() {
    dp.resize(n);
    dfs1(0), dfs2(0);
}
```

10.4 all longest path top sort

```
// all longest path in DAG
// 1. topological sort
vector<int> in(n, 0);
for (int i = 0; i < m; ++i) {
    int a, b, w;
    cin >> a >> b >> w;
    adj[a].emplace_back(b, w);
    in[b]++;
}

vector<int> topo; // sequence of top sort
queue<int> q;
for (int i = 0; i < n; ++i) {
    if (in[i] == 0) {
        q.push(i);
    }
}
while (!q.empty()) {
    int pa = q.front();
    q.pop();
    topo.push_back(pa);
    for (auto& [child, w] : adj[pa]) {
        in[child]--;
        if (in[child] == 0) {
            q.push(child);
        }
    }
}

// all longest path
vector<int> dist(n, INT_MIN);
vector<vector<int>> parents(n);
dist[0] = process[0];

for (int u : topo) {
    for (auto& [v, w] : adj[u]) {
        if (dist[v] < dist[u] + process[v] +
             w) {
            dist[v] = dist[u] + process[v] +
                 w;
            parents[v] = {u};
        }
        else if (dist[v] == dist[u] +
             process[v] + w) {
            parents[v].push_back(u);
        }
    }
}

cout << dist[n - 1];
```

10.5 Heavy-light decomposition (HLD)

```
const int maxn = 2e5;
int n, q;
int val[maxn];
int id[maxn];
int sz[maxn];
int parent[maxn];
int depth[maxn];
int tp[maxn];
vector<int> adj[maxn];
int timer = 0;

int dfs_sz(int u, int p) {
    sz[u] = 1;
    parent[u] = p;

    // calculate size of each subtree
    for (int v : adj[u]) {
        if (v != p) {
            depth[v] = depth[u] + 1;
            sz[u] += dfs_sz(v, u);
        }
    }
    return sz[u];
}
```

```cpp
void dfs_hld(int u, int p, int top, SGT<int, MergeMax>& tree) {
    id[u] = timer++;
    tp[u] = top;
    tree.update(id[u], val[u]);

    // find the heavy child
    int h_v = -1, h_sz = -1;
    for (int v : adj[u]) {
        if (v != p) {
            if (h_sz < sz[v]) {
                h_sz = sz[v];
                h_v = v;
            }
        }
    }
    if (h_v == -1) {
        return;
    }

    // continue the chain
    dfs_hld(h_v, u, top, tree);
    // start a new chain for each light child
    for (int v : adj[u]) {
        if (v != p && v != h_v) {
            dfs_hld(v, u, v, tree);
        }
    }
}

int path(int x, int y, SGT<int, MergeMax>& tree) {
    int ans = 0;
    // move x and y to their top nodes until they are in the same chain
    while (tp[x] != tp[y]) {
        if (depth[tp[x]] < depth[tp[y]])
            swap(x, y);
        ans = max(ans, tree.query(id[tp[x]], id[x] + 1));
        x = parent[tp[x]];
    }
    if (depth[x] > depth[y]) swap(x, y);
    ans = max(ans, tree.query(id[x], id[y] + 1));
    return ans;
}

void solve() {
    cin >> n >> q;
    SGT<int, MergeMax> tree(n, -1);

    for (int i = 0; i < n; ++i) {
        cin >> val[i];
    }
    for (int i = 0; i < n - 1; ++i) {
        int a, b;
        cin >> a >> b;
        a--, b--;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }

    dfs_sz(0, 0);
    dfs_hld(0, 0, 0, tree);

    cerr << "tp: ";
    for (int i = 0; i < n; ++i) {
        cerr << tp[i] << " ";
    }
    cerr << "\n";

    while (q--) {
        int mode;
        cin >> mode;
        if (mode == 1) {
            int s, x;
            cin >> s >> x;
            s--;
            val[s] = x;
            tree.update(id[s], val[s]);
        }
        else {
            int a, b;
            cin >> a >> b;
            cout << path(a - 1, b - 1, tree) << " ";
        }
    }
}
```

10.6 get kth ancestor

```cpp
void dfs(int u, int p = -1) {
    up[0][u] = (p == -1 ? u : p);
    for (int v : adj[u]) {
        if (v != p) {
            dfs(v, u);
        }
    }
}

int get_kth_ancestor(int x, int k) {
    for (int i = 0; i <= LOG; ++i) {
        if (k & (1LL<<i)) {
            x = up[i][x];
        }
    }
    return (x == 0 ? -1 : x);
}

void solve() {
    int n, q;
    cin >> n >> q;

    adj[0].push_back(1);
    for (int i = 2; i <= n; ++i) {
        int u;
        cin >> u;
        adj[u].push_back(i);
    }

    dfs(0, -1);

    for (int k = 1; k <= LOG; ++k) {
        for (int v = 1; v <= n; ++v) {
            up[k][v] = up[k - 1][up[k - 1][v]];
        }
    }

    while (q--) {
        int x, k;
        cin >> x >> k;
        cout << get_kth_ancestor(x, k) << "\n";
    }
}
```

10.7 path maximum edge between u, v in tree

```cpp
int depth[maxn];
int up[LOG][maxn];
int maxEdge[LOG][maxn];

void dfs(int u, int p = -1, int w = 0) {
    up[0][u] = (p == -1 ? u : p);
    maxEdge[0][u] = (p == -1 ? 0 : w);
    for (auto &[v, rw] : adj[u]) {
        if (v != p) {
            depth[v] = depth[u] + 1;
            dfs(v, u, rw);
        }
    }
}

int lca_weight(int a, int b) {
    if (depth[a] < depth[b]) swap(a, b);
    int diff = depth[a] - depth[b];
    int ans = 0;

    for (int k = 0; diff; ++k) {
        if (diff & 1) {
            ans = max(ans, maxEdge[k][a]);
            a = up[k][a];
        }
        diff >>= 1;
    }
    if (a == b) {
        return ans;
    }
    for (int k = LOG - 1; k >= 0; --k) {
        if (up[k][a] != up[k][b]) {
            ans = max(ans, maxEdge[k][a]);
            ans = max(ans, maxEdge[k][b]);
            a = up[k][a];
            b = up[k][b];
        }
    }

    ans = max(ans, maxEdge[0][a]);
    ans = max(ans, maxEdge[0][b]);
    return ans;
}

void solve() {
    dfs(0, -1, 0);
    for (int k = 1; k < LOG; ++k) {
        for (int v = 0; v < n; ++v) {
            up[k][v] = up[k - 1][up[k - 1][v]];
            maxEdge[k][v] = max(maxEdge[k - 1][v], maxEdge[k - 1][up[k - 1][v]]);
        }
    }
    int path_max = lca_weight(0, 1);
}
```

10.8 binary jumping

```cpp
const int LOG = 32;
const int maxn = 2e5 + 2;

int depth[maxn];
int up[LOG][maxn];
vector<int> adj[maxn];

void dfs(int u, int p = -1) {
    up[0][u] = (p == -1 ? u : p);
    for (int v : adj[u]) {
        if (v != p) {
            depth[v] = depth[u] + 1;
            dfs(v, u);
        }
    }
}


void solve() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    cin >> n >> q;

    for (int i = 2; i <= n; ++i) {
        int u;
        cin >> u;
        adj[u].push_back(i);
    }

    dfs(1, -1);  // root = 1

    for (int k = 1; k < LOG; ++k) {
        for (int v = 1; v <= n; ++v) {
            up[k][v] = up[k - 1][up[k - 1][v]];
        }
    }

    while (q--) {
        int a, b;
        cin >> a >> b;
        cout << lca(a, b) << "\n";
    }
}
```

10.9 tree diameter

```cpp
int diam = 0;

int dfs(int u, int p = -1) {
    int mx = 0;
    for (int v : adj[u]) {
        if (v != p) {
            int len = 1 + dfs(v, u);
            diam = max(diam, mx + len);
            mx = max(mx, len);
        }
    }
    return mx;
}
```

10.10 all longest path

```cpp
int fir[maxn]; // length of the longest downward path from u into its subtree.
int sec[maxn]; // length of the second longest downward path from u
int res[maxn];

void dfs1(int u, int p) {
    for (int v : adj[u]) {
        if (v != p) {
            dfs1(v, u);
            if (fir[v] + 1 > fir[u]) {
                sec[u] = fir[u];
                fir[u] = fir[v] + 1;
            }
            else if (fir[v] + 1 > sec[u]) {
                sec[u] = fir[v] + 1;
            }
        }
    }
}

// to_p: the best path length that comes from the parent' s side
void dfs2(int u, int p, int to_p) {
    res[u] = max(to_p, fir[u]);

    for (int v : adj[u]) {
        if (v != p) {
            if (fir[v] + 1 == fir[u]) {
                dfs2(v, u, max(to_p, sec[u]) + 1);
            }
            else {
                dfs2(v, u, res[u] + 1);
            }
        }
    }
}

// usage
dfs1(1, 0);
dfs2(1, 0, 0);
// Now res[i] is the maximum distance from node i to any other node
for (int i = 1; i <= n; i++) {
    cout << res[i] << " ";
}
```

10.11 tree diameter (len,end)

```cpp
array<int, 2> dfs(int u, int w = -1) {
    array<int, 2> mx{0, u};  // {length, farthest leaf}
    for (auto& v : adj[u]) {
        if (v == w) continue;
        auto [len, leaf]{dfs(v, u)};
        mx = max(mx, {len + 1, leaf});
    }
    return mx;
}

array<int, 3> tree_diameter(int a = 0) {
    auto b{dfs(a)[1]};      // farthest node from 'a'
    auto [l, c]{dfs(b)};    // farthest node from 'b'
    return {l, b, c};       // {diameter length, endpoint1, endpoint2}
}
```

10.12 minimum distance in functional graph

```cpp
// Given a functional graph (each node has exactly one outgoing edge), answer
// queries about the minimum distance between two nodes, or determine that
// it's impossible to get from one to the other.
// There are three cases:
// 1. Both in Tree
// 2. Both in Cycle
// 3. One in Tree, one in Cycle

int main() {
  int planet_num;
  int query_num;
  std::cin >> planet_num >> query_num;
  vector<int> next(planet_num);
  vector<vector<int>> before(planet_num);
  for (int p = 0; p < planet_num; p++) {
    std::cin >> next[p];
    next[p]--;
    before[next[p]].push_back(p);
  }

  /*
   * -2 = We haven't even got to processing this planet yet.
   * -1 = This node is part of a tree.
   * >= 0: the ID of the cycle the planet belongs to.
   */
  vector<int> cycle_id(planet_num, -2);
  // Each map, given a planet #, returns the index of that planet in the
  // cycle.
  vector<std::map<int, int>> cycles;
  for (int p = 0; p < planet_num; p++) {
    if (cycle_id[p] != -2) { continue; }
    vector<int> path{p};
    int at = p;
    while (cycle_id[next[at]] == -2) {
      at = next[at];
      cycle_id[at] = -3;  // Leave breadcrumbs for this iteration
      path.push_back(at);
    }

    std::map<int, int> cycle;
    bool in_cycle = false;
    for (int i : path) {
      in_cycle = in_cycle || i == next[at];
      if (in_cycle) { cycle[i] = cycle.size(); }
      cycle_id[i] = in_cycle ? cycles.size() : -1;
    }
    cycles.push_back(cycle);
  }

  /*
   * Precalculate the distance from each planet to its cycle with BFS.
   * (cyc_dist[p] = 0) if p is part of a cycle.
   */
  vector<int> cyc_dist(planet_num);
  for (int p = 0; p < planet_num; p++) {
    // Check if this planet is part of a cycle.
    if (cycle_id[next[p]] == -1 || cycle_id[p] != -1) { continue; }
    cyc_dist[p] = 1;
    vector<int> stack(before[p]);
    while (!stack.empty()) {
      int curr = stack.back();
      stack.pop_back();
      cyc_dist[curr] = cyc_dist[next[curr]] + 1;
      stack.insert(stack.end(), before[curr].begin(), before[curr].end());
    }
  }

  // Initialize the binary jumping arrays.
  int log2 = std::ceil(std::log2(planet_num));
  vector<vector<int>> pow2_ends(planet_num, vector<int>(log2 + 1));
  for (int p = 0; p < planet_num; p++) { pow2_ends[p][0] = next[p]; }
  for (int i = 1; i <= log2; i++) {
    for (int p = 0; p < planet_num; p++) {
      pow2_ends[p][i] = pow2_ends[pow2_ends[p][i - 1]][i - 1];
    }
  }

  /*
   * Given a starting planet & dist, returns the planet we end up at
   * if we use the teleporter dist times.
   */
  auto advance = [&](int pos, int dist) {
    for (int pow = log2; pow >= 0; pow--) {
      if ((dist & (1 << pow)) != 0) { pos = pow2_ends[pos][pow]; }
    }
    return pos;
  };
  for (int q = 0; q < query_num; q++) {
    int u, v;  // going from u to v
    std::cin >> u >> v;
    u--;
    v--;
    if (cycle_id[pow2_ends[u][log2]] != cycle_id[pow2_ends[v][log2]]) {
      cout << -1 << '\n';
      continue;
    }
    if (cycle_id[u] != -1 || cycle_id[v] != -1) {
      if (cycle_id[v] == -1 && cycle_id[u] != -1) {
        cout << -1 << '\n';
        continue;
      }
      // Handle the 2nd & 3rd cases at the same time.
      int dist = cyc_dist[u];
      int u_cyc = advance(u, cyc_dist[u]);
      // The root of u's tree

      std::map<int, int> &cyc = cycles[cycle_id[u_cyc]];  // u and v's cycle
      int u_ind = cyc[u_cyc];
      int v_ind = cyc[v];
      int diff = u_ind <= v_ind ? v_ind - u_ind : cyc.size() - (u_ind - v_ind);
      cout << dist + diff << '\n';
    } else {
      if (cyc_dist[v] > cyc_dist[u]) {
        cout << -1 << '\n';
        continue;
      }
      int diff = cyc_dist[u] - cyc_dist[v];
      cout << (advance(u, diff) == v ? diff : -1) << '\n';
    }
  }
}
```

10.13 union length of two tree paths

```cpp
/*
Problem: Given a tree and queries (a,b,c,d), find the size of the union of nodes on paths a↔b and c↔d.
```

```
Let P1 be the nodes on the path a↔b and P2 the nodes on c↔d.
We want |P1 ⨁ P2| = |P1| + |P2| - |P1 ∩ P2|.
|P1| = dist(a,b) + 1, |P2| = dist(c,d) + 1. So the only hard part is |P1 ∩ P2|.

Key facts for trees:

The intersection of two simple paths is itself either empty or a simple path.

Any endpoint of that intersection must be one of the LCAs among endpoint pairs of the two paths.


So for each query:

compute the 6 candidate LCAs,

keep those candidates that lie on both paths (test with distances),

if none → intersection size = 0; if one → intersection size = 1;
otherwise take the two kept candidates with greatest depth (they are the endpoints of the intersection)
and intersection size = dist(p,q) + 1.
*/

const int LOG = 32;

const int maxn = 1e5;
int depth[maxn];
vector<int> adj[maxn];
int up[LOG + 1][maxn];
int n, q;

void dfs(int u, int p = -1) {
    up[0][u] = (p == -1 ? u : p);
    for (int v : adj[u]) {
        if (v != p) {
            depth[v] = depth[u] + 1;
            dfs(v, u);
        }
    }
}

int dist(int a, int b) {
    int c = lca(a, b);
    return depth[a] + depth[b] - 2 * depth[c];
}

// check z on path u-v
bool on_path(int z, int u, int v) {
    return dist(u, z) + dist(z, v) == dist(u, v);
}

void solve() {
    cin >> n >> q;
    for (int i = 0; i < n - 1; ++i) {
        int a, b;
        cin >> a >> b;
        a--, b--;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }
    memset(depth, 0, sizeof(depth));
    dfs(0, -1);

    for (int k = 1; k < LOG; ++k) {
        for (int v = 0; v < n; ++v) {
            up[k][v] = up[k - 1][up[k - 1][v]];
        }
    }

    while (q--) {
        int a, b, c, d;
        cin >> a >> b >> c >> d;
        a--, b--, c--, d--;
        int len1 = dist(a, b) + 1;
        int len2 = dist(c, d) + 1;

        array<int, 6> cand = {
            lca(a, b), lca(c, d),
            lca(a, c), lca(a, d),
            lca(b, c), lca(b, d)
        };

        vector<int> good;
        for (int z : cand) {
            if (on_path(z, a, b) && on_path(z, c, d)) {
                good.push_back(z);
            }
        }

        int inter = 0;
        if (good.size() == 1) {
            inter = 1;
        }
        else if (good.size() > 1) {
            sort(good.begin(), good.end(), [&](int x, int y) {
                return depth[x] > depth[y];
            });
            int p = good[0], qn = good[1];
            inter = dist(p, qn) + 1;
        }
        cout << (len1 + len2 - inter) << "\n";
    }
}
```

10.14 Centroid decomposition

```
// Centroid Decomposition
// Problem: Given a tree with N nodes (1-indexed) and M queries. Each query is one of the following two types:
// 1 v: Paint node v red.
// 2 v: Print the minimal distance between node v and any red node.
// Initially, only node 1 is red.
// Constraints: 1 <= N, M <= 10^5
// Time complexity: O((N + M) log N)

vector<vector<int>> adj;
vector<int> subtree_size;
// min_dist[v] = the minimal distance between v and a red node
vector<int> min_dist;
vector<bool> is_removed;
vector<vector<pii>> ancestors;

int get_subtree_size(int u, int p = -1) {
    subtree_size[u] = 1;
    for (int v : adj[u]) {
        if (v == p || is_removed[v])
            continue;
        subtree_size[u] += get_subtree_size(v, u);
    }
    return subtree_size[u];
}

int get_centroid(int u, int tree_size, int p = -1) {
    for (int v : adj[u]) {
        if (v == p || is_removed[v])
            continue;
        if (subtree_size[v] * 2 > tree_size) {
            return get_centroid(v, tree_size, u);
        }
    }
    return u;
}

// Calculate the distance between current `u` and the `centroid` it belongs.
void get_dist(int u, int centroid, int p = -1, int cur_dist = 1) {
    for (int v : adj[u]) {
        if (v == p || is_removed[v])
            continue;
        cur_dist++;
        get_dist(v, centroid, u, cur_dist);
        cur_dist--;
    }
    ancestors[u].emplace_back(centroid, cur_dist);
}

void build_centroid_decomp(int u = 0) {
    int centroid = get_centroid(u, get_subtree_size(u));

    // For all nodes in the subtree rooted at `centroid`, calculate their distance to the centroid
    for (int v : adj[centroid]) {
        if (is_removed[v]) continue;
        get_dist(v, centroid, centroid);
    }

    is_removed[centroid] = true;
    for (int v : adj[centroid]) {
        if (is_removed[v]) continue;
        build_centroid_decomp(v);
    }
}

// Paint `node` red by updating all of its ancestors' minimal distances to a red node
void paint(int u) {
    for (auto &[ancestor, dist] : ancestors[u]) {
        min_dist[ancestor] = min(min_dist[ancestor], dist);
    }
    min_dist[u] = 0;
}

// Print the minimal distance between `u` to a red node.
void query(int u) {
    int ans = min_dist[u];
    for (auto &[ancestor, dist] : ancestors[u]) {
        if (!dist) continue;
        ans = min(ans, dist + min_dist[ancestor]);
    }
    cout << ans << "\n";
}

void solve() {
    int N, M;
    cin >> N >> M;

    adj.assign(N, vector<int>());
    for (int i = 0; i < N - 1; ++i) {
        int a, b;
        cin >> a >> b;
        a--, b--;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }

    subtree_size.assign(N, 0);
    ancestors.assign(N, vector<pii>());
    is_removed.assign(N, false);
    build_centroid_decomp();

    min_dist.assign(N, INF);
    paint(0);
    for (int i = 0; i < M; ++i) {
        int t, v;
        cin >> t >> v;
        v--;
        if (t == 1) {
            paint(v);
        }
        else {
            query(v);
        }
    }
}
```

11 default

11.1 debug

```
#ifdef DEBUG
#define dbg(...) {\
  fprintf(stderr,"%s - %d : (%s) = ",
       __PRETTY_FUNCTION__,__LINE__,#
       __VA_ARGS__);\
  _DO(__VA_ARGS__);\
}
template<typename I> void _DO(I&&x){cerr<<x
    <<endl;}
template<typename I,typename...T> void _DO(I
    &&x,T&&...tail){cerr<<x<<", ";_DO(tail
    ...);}
#else
#define dbg(...)
#endif
```

11.2 template

```
// alias g++='g++ -std=c++14 -fsanitize=
    undefined -Wall -Wextra -Wshadow -D
    LOCAL'

#include <bits/stdc++.h>
using namespace std;

#ifdef LOCAL
#include "../debug.h"
#else
#pragma GCC optimize("O3,unroll-loops")
#pragma GCC target("avx2,bmi,bmi2,lzcnt,
    popcnt")
#define debug(...) ((void)0)
#define orange(...) ((void)0)
#endif

#define int long long
#define SPEEDY ios_base::sync_with_stdio(
    false); cin.tie(0); cout.tie(0);

void solve() {

}

signed main() {
    SPEEDY;
    return 0;
}
```

11.3 vimrc

```
set nu
set ts=4
set shiftwidth=4
set autoindent smartindent
set cindent

imap jk <ESC>
inoremap {<CR> {<CR>}<ESC>ko
```

12 other

12.1 Nim game

```
a1^a2^a3^...^an != 0 ? A win : B win
```

12.2 找小於 n 所有出現的 1 數量

```
current == 0 higher * factor
current == 1 higher * factor + lower + 1
other current (higher + 1) * factor
```

13 other language

13.1 python heap

```
import heapq

heap = [7,1,2,2]
heapq.heapify(heap)
print(heap) # [1, 2, 2, 7]
heapq.heappush(heap, 5)
print(heap) # [1, 2, 2, 7, 5]
print(heapq.heappop(heap)) # 1
print(heap) # [2, 2, 5, 7]
```

13.2 java

13.2.1 文件操作

```
import java.io.*;
import java.util.*;
import java.math.*;
import java.text.*;

public class Main{
  public static void main(String args[]){
      throws FileNotFoundException,
      IOException
    Scanner sc = new Scanner(new FileReader(
        "a.in"));
    PrintWriter pw = new PrintWriter(new
        FileWriter("a.out"));
    int n,m;
    n=sc.nextInt();
    m=sc.nextInt();

    for(ci=1; ci<=c; ++ci){
      pw.println("Case #"+ci+": easy for
          output");
    }

    pw.close();
    sc.close();
  }
}
```

13.2.2 优先队列

```
PriorityQueue queue = new PriorityQueue( 1,
     new Comparator(){
  public int compare( Point a, Point b ){
  if( a.x < b.x || a.x == b.x && a.y < b.y )
    return -1;
  else if( a.x == b.x && a.y == b.y )
    return 0;
  else return 1;
  }
});
```

13.2.3 Map

```
Map map = new HashMap();
map.put("sa","dd");
String str = map.get("sa").toString;

for(Object obj : map.keySet()){
  Object value = map.get(obj );
}
```

13.2.4 sort

```
static class cmp implements Comparator{
  public int compare(Object o1,Object o2){
  BigInteger b1=(BigInteger)o1;
  BigInteger b2=(BigInteger)o2;
  return b1.compareTo(b2);
  }
}
public static void main(String[] args)
    throws IOException{
  Scanner cin = new Scanner(System.in);
  int n;
  n=cin.nextInt();
  BigInteger[] seg = new BigInteger[n];
  for (int i=0;i<n;i++)
  seg[i]=cin.nextBigInteger();
  Arrays.sort(seg,new cmp());
}
```

13.3 python output

```
hello = 'Hello'
world = 7122
print(f'{hello} {world}') # Hello 7122

import math
print(f'PI is approximately {math.pi:.3f}.')
# PI is approximately 3.142.

print('AAA {} BBB "{}!"'.format('Jin', 'Kela
    '))
# AAA Jin BBB "Kela!"

hello = 'hello, world\n'
hellos = repr(hello)
print(hellos) # 'hello, world\n'

x = 32.5
y = 40000
print(repr((x, y, ('spam', 'eggs'))))
# "(32.5, 40000, ('spam', 'eggs'))"

x = 7
print(eval('3 * x')) # 21
```

13.4 python 大數因數分解

```
# 大數因數分解 (使用 Pollard's Rho 與 Miller
    -Rabin)
import sys, random
from math import gcd

# Miller-Rabin 檢定 ( 機率性質數判定 )
def is_probable_prime(n, k=12):
    if n < 2:
        return False
    # 先檢查一些小質數
    small_primes =
        [2,3,5,7,11,13,17,19,23,29]
    for p in small_primes:
        if n % p == 0:
            return n == p
    # 把 n-1 寫成 d * 2^s
    d = n - 1
    s = 0
    while d % 2 == 0:
        d //= 2
        s += 1
    # 重複 k 次隨機測試
    for _ in range(k):
        a = random.randrange(2, n - 1)  # 隨
            機挑一個測試基數
        x = pow(a, d, n)
        if x == 1 or x == n - 1:
            continue
        composite = True
        for __ in range(s - 1):
            x = pow(x, 2, n)
            if x == n - 1:
                composite = False
```

```
                break
        if composite:
            return False
    return True

# Pollard's Rho 演算法（找非平凡因數）
def pollards_rho(n):
    if n % 2 == 0:
        return 2
    if n % 3 == 0:
        return 3
    # 隨機多項式 (x^2 + c) mod n
    while True:
        c = random.randrange(1, n-1)     #
            隨機挑選常數 c
        x = random.randrange(2, n-1)     #
            起始點
        y = x
        d = 1
        while d == 1:
            x = (pow(x, 2, n) + c) % n    # x
                -> f(x)
            y = (pow(y, 2, n) + c) % n    # y
                -> f(f(y))，走兩步
            y = (pow(y, 2, n) + c) % n
            d = gcd(abs(x - y), n)        #
                計算兩者差的 gcd
            if d == n:                    #
                失敗就重試
                break
        if d > 1 and d < n:               #
            找到非平凡因數
            return d

# 遞迴分解
def factor(n, out):
    if n == 1:
        return
    if is_probable_prime(n):
        out.append(n)
    else:
        d = pollards_rho(n)
        while d is None or d == n:  # 偶爾失
            敗就重試
            d = pollards_rho(n)
        factor(d, out)
        factor(n // d, out)

def main():
    data = sys.stdin.read().strip().split()
    if not data:
        return
    # 每個 token 當作一個數字
    for token in data:
        try:
            n = int(token)
        except:
            continue
        if n <= 1:
            print(n)
            continue
        facs = []
        factor(n, facs)
        facs.sort()
        # 輸出因數
        print(" ".join(str(x) for x in facs)
            )

if __name__ == "__main__":
    random.seed()  # 使用系統時間作為隨機種
        子
    main()
```

13.5 decimal

```
# 使用 decimal 模組來處理高精度小數運算
from decimal import *
setcontext(Context(prec=MAX_PREC, Emax=
    MAX_EMAX, rounding=ROUND_FLOOR))
print(Decimal(input()) * Decimal(input()))

# 將小數轉成分數，方便做近似或理論分析，且可
    以限制分母大小。
from fractions import Fraction
Fraction('3.14159').limit_denominator(10).
    numerator # 22

# 設定精確度
from decimal import Decimal, getcontext

# 精確位數設定
getcontext().prec = 70

n = 100
# 指定n為高精確度的物件
n = Decimal(n)
n /= 7
print(n)

# 將小數轉成分數
from fractions import Fraction

n = 1.5654
# 建立一個轉換物件
n = Fraction(n)

print(n)
```

13.6 python 大數排序

```
# 大數排序
# line one n : 多少數字
# next line : 依序輸入每行一個
# sort : sort + lambda
from sys import stdin

data = stdin.read().splitlines()

i = 0

while(i < len(data)):
    T = int(data[i].strip())
    i += 1
    temp = [int(data[i + index].strip()) for
         index in range(T)]
    i += T
    temp = sorted(temp, key = lambda x : (x)
        )
    for ele in temp:
        print(ele)
```

13.7 python 大數計算 2

```
# 單行輸入
# format : n1, operation, n2
from sys import stdin

data = stdin.read().splitlines()

limit = len(data)
i = 0

while(i < limit):
    a, operation, b = map(str, data[i].split
        ())
    a, b = int(a), int(b)
    i += 1
    if(operation == '+'):
        print(int(a + b))
    elif(operation == '-'):
        print(int(a - b))
    elif(operation == '*'):
        print(int(a * b))
    else:
        print(int(a // b))
```

13.8 python input

```
ans = sum(map(float, input().split()))
# input: 1.1 2.2 3.3 4.4 5.5
print(ans) # 16.5

(n, m) = map(int, input().split()) # 300 200
print(n * m) # 60000

Arr = list(map(int, input().split()))
# input: 1 2 3 4 5
print(Arr) # [1, 2, 3, 4, 5]
```

13.9 python 大數計算

```
# 讀取多行輸入
# line one first number
# line two operation
# line three second number
from sys import stdin

data = stdin.read().splitlines()

limit = len(data)
i = 0
while(i < limit):
    a = int(data[i].strip())
    i += 1
    operation = data[i].strip()
    i += 1
    b = int(data[i].strip())
    i += 1
    if(operation == '*'):
        print(int(a * b))
    else:
        print(int(a / b))
```

13.10 base-6 pow

```
import sys

sys.setrecursionlimit(2000000)

def solve():
    S = sys.stdin.readline().strip()
    try:
        N = int(S)
    except ValueError:
        N = 0
    if N == 0:
        print(1)
        return
    low = 1
    high = 700000  # Safely larger than
        500000 * 1.3
    ans = high + 1
    while low <= high:
        L_mid = (low + high) // 2
        if L_mid <= 0:  # Avoid 6^0 or 6^
            negative
            low = L_mid + 1
            continue
        val = pow(6, L_mid)
        if val > N:
            ans = L_mid
            high = L_mid - 1
        else:
            low = L_mid + 1
    print(ans)
solve()
```

14 zformula

14.1 formula

14.1.1 Pick 公式

給定頂點坐標均是整點的簡單多邊形，面積 = 內部格點數 + 邊上格點數/2-1

14.1.2 圖論

1. 對於平面圖，$F = E - V + C + 1$，C 是連通分量數
2. 對於平面圖，$E \leq 3V - 6$
3. 對於連通圖 G，最大獨立點集的大小設為 I(G)，最大匹配大小設為 M(G)，最小點覆蓋設為 Cv(G)，最小邊覆蓋設為 Ce(G)。對於任意連通圖：
   (a) $I(G) + Cv(G) = |V|$
   (b) $M(G) + Ce(G) = |V|$
4. 對於連通二分圖：
   (a) $I(G) = Cv(G)$
   (b) $M(G) = Ce(G)$

14.1.3 學長公式

1. $\sum_{d|n} \phi(n) = n$
2. Harmonic series $H_n = \ln(n) + \gamma + 1/(2n) - 1/(12n^2) + 1/(120n^4)$
3. $\gamma = 0.57721566490153286060651209008240243104215$
4. 選轉矩陣 $M(\theta) = \begin{pmatrix} cos\theta & -sin\theta \\ sin\theta & cos\theta \end{pmatrix}$

14.1.4 基本數論

1. $\sum_{d|n} \mu(n) = [n == 1]$
2. $g(m) = \sum_{d|m} f(d) \Leftrightarrow f(m) = \sum_{d|m} \mu(d) \times g(m/d)$
3. $\sum_{i=1}^{n} \sum_{j=1}^{m}$ 互質數量 $= \sum \mu(d) \lfloor \frac{n}{d} \rfloor \lfloor \frac{m}{d} \rfloor$
4. $\sum_{i=1}^{n} \sum_{j=1}^{n} lcm(i,j) = n \sum_{d|n} d \times \phi(d)$

14.1.5 排組公式

1. k 卡特蘭 $\frac{C_n^{kn}}{n(k-1)+1}$，$C_m^n = \frac{n!}{m!(n-m)!}$
2. $H(n,m) \cong x_1 + x_2 \ldots + x_n = k, num = C_k^{n+k-1}$
3. Stirling number of 2^{nd},n 人分 k 組方法數目
   (a) $S(0,0) = S(n,n) = 1$
   (b) $S(n,0) = 0$
   (c) $S(n,k) = kS(n-1,k) + S(n-1,k-1)$
4. Bell number,n 人分任意多組方法數目
   (a) $B_0 = 1$
   (b) $B_n = \sum_{i=0}^{n} S(n,i)$
   (c) $B_{n+1} = \sum_{k=0}^{n} C_k^n B_k$
   (d) $B_{p+n} \equiv B_n + B_{n+1} mod p$, p is prime
   (e) $B_{p^m+n} \equiv mB_n + B_{n+1} mod p$, p is prime
   (f) From B_0 : 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, 115975
5. Derangement, 錯排, 沒有人在自己位置上
   (a) $D_n = n!(1 - \frac{1}{1!} + \frac{1}{2!} - \frac{1}{3!} \ldots + (-1)^n \frac{1}{n!})$
   (b) $D_n = (n-1)(D_{n-1} + D_{n-2}), D_0 = 1, D_1 = 0$
   (c) From D_0 : 1, 0, 1, 2, 9, 44, 265, 1854, 14833, 133496
6. Binomial Equality
   (a) $\sum_k \binom{r}{m+k}\binom{s}{n-k} = \binom{r+s}{m+n}$
   (b) $\sum_k \binom{l}{m+k}\binom{s}{n+k} = \binom{l+s}{l-m+n}$
   (c) $\sum_k \binom{l}{m+k}\binom{s+k}{n}(-1)^k = (-1)^{l+m}\binom{s-m}{n-l}$
   (d) $\sum_{k\leq l} \binom{l-k}{m}\binom{s}{k-n}(-1)^k = (-1)^{l+m}\binom{s-m-1}{l-n-m}$
   (e) $\sum_{0\leq k\leq l} \binom{l-k}{m}\binom{q+k}{n} = \binom{l+q+1}{m+n+1}$
   (f) $\binom{r}{k} = (-1)^k\binom{k-r-1}{k}$
   (g) $\binom{r}{m}\binom{m}{k} = \binom{r}{k}\binom{r-k}{m-k}$
   (h) $\sum_{k\leq n} \binom{r+k}{k} = \binom{r+n+1}{n}$
   (i) $\sum_{0\leq k\leq n} \binom{k}{m} = \binom{n+1}{m+1}$
   (j) $\sum_{k\leq m} \binom{m+r}{k}x^k y^k = \sum_{k\leq m} \binom{-r}{k}(-x)^k(x+y)^{m-k}$

14.1.6 冪次, 冪次和

1. $a^{b\%P} = a^{b\%\varphi(p)+\varphi(p)}, b \geq \varphi(p)$
2. $1^3 + 2^3 + 3^3 + \ldots + n^3 = \frac{n^4}{4} + \frac{n^3}{2} + \frac{n^2}{4}$
3. $1^4 + 2^4 + 3^4 + \ldots + n^4 = \frac{n^5}{5} + \frac{n^4}{2} + \frac{n^3}{3} - \frac{n}{30}$
4. $1^5 + 2^5 + 3^5 + \ldots + n^5 = \frac{n^6}{6} + \frac{n^5}{2} + \frac{5n^4}{12} - \frac{n^2}{12}$
5. $0^k + 1^k + 2^k + \ldots + n^k = P(k), P(k) = \frac{(n+1)^{k+1} - \sum_{i=0}^{k-1} C_i^{k+1} P(i)}{k+1}, P(0) = n+1$
6. $\sum_{k=0}^{m-1} k^n = \frac{1}{n+1} \sum_{k=0}^{n} C_k^{n+1} B_k m^{n+1-k}$
7. $\sum_{j=0}^{m} C_j^{m+1} B_j = 0, B_0 = 1$
8. 除了 $B_1 = -1/2$，剩下的奇數項都是 0
9. $B_2 = 1/6, B_4 = -1/30, B_6 = 1/42, B_8 = -1/30, B_{10} = 5/66, B_{12} = -691/2730, B_{14} = 7/6, B_{16} = -3617/510, B_{18} = 43867/798, B_{20} = -174611/330,$

14.1.7 循環小數轉分數

1. 若 $x = 0.\overline{a}$，則
$$x = \frac{a}{\underbrace{99\ldots9}_{k \text{ digits}}}$$
其中 a 為循環節，k 為循環節的位數。
2. 例子：
$$0.\overline{37} = \frac{37}{99}$$
$$0.\overline{5} = \frac{5}{9}$$
3. 純循環小數：若 $x = 0.\overline{a}$，其中 a 為循環節、長度為 k，
$$x = \frac{a}{\underbrace{99\ldots9}_{k \text{ digits}}}$$
例：
$$0.\overline{37} = \frac{37}{99}, \quad 0.\overline{5} = \frac{5}{9}$$
4. 混循環小數：若 $x = 0.b\overline{a}$，其中 b 為前綴、長度 m，a 為循環節、長度 k，
$$x = \frac{(b \cdot 10^k + a) - b}{10^m(10^k - 1)}$$
例：
$$0.12\overline{3} = \frac{(12 \cdot 10^1 + 3) - 12}{10^2(10^1 - 1)} = \frac{123 - 12}{100 \cdot 9} = \frac{111}{900} = \frac{37}{300}$$

14.1.8 常見級數與組合公式

1. 平方和公式：
$$1^2 + 2^2 + \cdots + n^2 = \frac{n(n+1)(2n+1)}{6}$$
$$4^2 + 6^2 + \cdots + (2n)^2 = \frac{(2n)(n+1)(2n+1)}{3}$$
$$1^2 + 3^2 + \cdots + (2n+1)^2 = \frac{n(2n-1)(2n+1)}{3}$$
2. 立方和公式：
$$1^3 + 2^3 + \cdots + n^3 = \frac{n^4 + 2n^3 + n^2}{4}$$
3. 四次方和公式：
$$1^4 + 2^4 + \cdots + n^4 = \frac{6n^5 + 15n^4 + 10n^3 - n}{30}$$
4. 五次方和公式：
$$1^5 + 2^5 + \cdots + n^5 = \frac{2n^6 + 6n^5 + 5n^4 - n^2}{12}$$
5. 六次方和公式：
$$1^6 + 2^6 + \cdots + n^6 = \frac{6n^7 + 21n^6 + 21n^5 - 7n^3 + n}{42}$$
6. 七次方和公式：
$$1^7 + 2^7 + \cdots + n^7 = \frac{3n^8 + 12n^7 + 14n^6 - 7n^4 + 2n^2}{24}$$
7. 八次方和公式：
$$1^8 + 2^8 + \cdots + n^8 = \frac{10n^9 + 45n^8 + 60n^7 - 42n^5 + 20n^3 - 3n}{90}$$
8. 九次方和公式：
$$1^9 + 2^9 + \cdots + n^9 = \frac{2n^{10} + 10n^9 + 15n^8 - 14n^6 + 10n^4 - 3n^2}{20}$$
9. 十次方和公式：
$$1^{10} + 2^{10} + \cdots + n^1 = \frac{6n^{11} + 33n^{10} + 55n^9 - 66n^7 + 66n^5 - 33n^3 + 5r}{66}$$
10. 等比級數：
$$S = a \cdot \frac{r^n - 1}{r - 1}$$
11. 二項式係數恆等式：
$$\binom{n}{0} + \binom{n}{1} + \cdots + \binom{n}{n} = 2^n$$
$$\binom{n}{1} + \binom{n}{2} + \cdots + \binom{n}{n} = 2^n - 1$$
$$\binom{n}{1} + \binom{n}{3} + \cdots + \binom{n}{n-k} = 2^{n-1}$$
12. 分配問題 (玩具分給小孩)：
   (a) n 個玩具，k 位小孩，可以有人沒拿到：
$$\binom{n+k-1}{n} = \binom{n+k-1}{k-1}$$
   (b) n 個玩具，k 位小孩，每個人至少一個：
$$\binom{n-1}{k-1}$$

14.1.9 位元運算

(a) 位元條件：
$$(x+k)\,\&\,(y+k) = 0$$
(b) 加法恆等式 (利用 XOR 與 AND)：
$$a + b = (a \oplus b) + 2 \cdot (a\,\&\,b)$$
(c) OR 與 AND 的關係：
$$a\,|\,b = a + b - (a\,\&\,b)$$
(d) 交換兩數：
$$a = a \oplus b, \quad b = a \oplus b, \quad a = a \oplus b$$
(e) 取得最低位的 1：
$$x\,\&\,(-x)$$
(f) 清除最低位的 1：
$$x\,\&\,(x-1)$$

14.1.10 除數與倍數

(a) LCM:

$$\operatorname{lcm}(a,b) = \frac{a \times b}{\gcd(a,b)}$$

(b) LCM (prime factorization):

$$\operatorname{lcm}(a,b) = \prod_{p \in \text{primes}} p^{\max(e_a, e_b)}$$

(c) GCD (prime factorization):

$$\gcd(a,b) = \prod_{p \in \text{primes}} p^{\min(e_a, e_b)}$$

(d) Counting divisors:

$$\text{If } n = \prod_{i=1}^{k} p_i^{e_i}, cnt \text{ is } \prod_{i=1}^{k} (e_i + 1)$$

14.1.11 數論公式

13. Bezout's identity：

$$ax + by = \gcd(a,b) \quad \text{（必定存在整數解 } x, y \text{）}$$

14. 模指數運算（冪的冪）:

$$a^{b^c} \equiv a^{\text{power\_mod}(b,c,\,\text{MOD}-1)} \pmod{\text{MOD}}$$

Pythagorean theorem：

$$(n^2 + m^2, 2nm, n^2 - m^2), \quad n > m > 0$$

Wilson's theorem：

$$(p-1)! \pmod p = n - 1 \quad \text{（} p \text{ 為質數）}$$

Catalan number：

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \frac{(2n)!}{(n+1)!n!}$$

海龍公式：

$$\text{Area} = \sqrt{s(s-a)(s-b)(s-c)}, \quad s = \frac{a+b+c}{2}$$

兩圓相交弦長：

$$\text{Length} = 2 \cdot \sqrt{\frac{(s-a)(s-b)}{s}}, \quad s = \frac{r_1 + r_2 + d}{2}$$

三角形內切圓半徑：

$$r = \frac{2 \cdot \text{Area}}{a+b+c}$$

三角形外接圓半徑：

$$R = \frac{abc}{4 \cdot \text{Area}}$$

三角形重心：

$$G = \left(\frac{x_1 + x_2 + x_3}{3}, \frac{y_1 + y_2 + y_3}{3}\right)$$

三角形垂心：

$$H = \left(\frac{x_1 \tan A + x_2 \tan B + x_3 \tan C}{\tan A + \tan B + \tan C}, \frac{y_1 \tan A + y_2 \tan B + y_3 \tan C}{\tan A + \tan B + \tan C}\right)$$

三角形內心：

$$I = \left(\frac{ax_1 + bx_2 + cx_3}{a+b+c}, \frac{ay_1 + by_2 + cy_3}{a+b+c}\right)$$

奇 (偶) 長度陣列子數量：

$$n = \frac{(i+1) * (n-i) + 1}{2}$$

15 Интернационал

15.1 保佑

```



//                    _oo0oo_
//                   o8888888o
//                   88" . "88
//                   (| -_- |)
//                   0\  =  /0
//                ___/`---'\___
//              .' \\|     |// '.
//             / \\|||  :  |||// \
//            / _||||| -:- |||||- \
//           |   | \\\  -  /// |   |
//           | \_|  ''\---/''  |_/ |
//           \  .-\__  '-'  ___/-. /
//         ___'. .'  /--.--\  `. .'___
//      ."" '<  `.___\_<|>_/___.' >' "".
//     | | :  `- \`.;`\ _ /`;.`/ - ` : | |
//     \  \ `_.   \_ __\ /__ _/   .-` /  /
// ===`-.____`.___ \_____/___.-`___.-'===
//                   `=---='
//
// ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
//          佛祖保佑            永無BUG







#     ┏━┓      ┏━┓
#  ┏━━┛ ┗━━━━━━┛ ┗━━┓
#  ┃                ┃
#  ┃      ━━━       ┃
#  ┃  ━┳┛       ┗┳━ ┃
#  ┃                ┃
#  ┃      ━┻━       ┃
#  ┃                ┃
#  ┗━━━┓       ┏━━━━┛
#      ┃       ┃
#      ┃       ┃
#      ┃       ┃
#      ┃       ┗━━━━━━━━━━━━━┓
#      ┃                     ┃
#      ┃                     ┣━┓
#      ┃                     ┏━┛
#      ┃                     ┃
#      ┗━┓  ┓ ┏━━━━━━━━━┳━━┓ ┏━┛
#        ┃ ━┫ ━┫        ┃ ━┫ ━┫
#        ┗━━┻━━┛        ┗━━┻━━┛
#              神獸保佑 永無BUG!



// ##         ##################
// ##         ##
// ##         ##
// ##         ##
// ##         ##
// ##         ##
// ##         ##
// ##         ##
// ################################
//            ##               ##
//            ##               ##
//            ##               ##
//            ##               ##
//            ##               ##
//            ##               ##
//            ##               ##
// ################            ##
//
//          元首保佑 永無BUG

//                 __
// _____ __ __   < @ \    _  __ _____
// \____\\_\\_)   /**|  (_//_//____/
//  \____\(_)\*\/****\/*/(_)/____/
//   \____\_/_\*\****/*/_\_/____/
//         u/u/u|****|u\u\u
//          u/u/|****|\u\u
//              |*||*|
//               |  |
//              /+--+\
//             ||  卐  ||
//              \\==//
//
//           神獸保佑 永無BUG
```

ACM ICPC Team Reference - BogoSort

Contents